



CST3990 Undergraduate Individual Project

Coursework Project – A Comparative Analysis of Similarity Measurement Techniques for an Academic Recommendation System

**Winter/Spring term
2025/2026**

Date of Submission: 12th April 2026

Student Name: Alvin Fadhili Mambo Muinde

Student ID Number: M00899452

Project Supervisor: Dr Gina Soopramanien

Declaration

I, *Alvin Fadhili Mambo Muinde*, declare that this thesis titled, “**A Comparative Analysis of Similarity Measurement Techniques for an Academic Recommendation System**” and the work presented in it are my own. I confirm that:

- This work was done wholly for a degree at Middlesex University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at Middlesex University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:



Date: *April 9th 2026*.

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, **Dr Gina Soopramanien**, for her guidance, feedback, and support throughout the development of this project. Her advice and encouragement were invaluable in shaping the direction of the work and improving the quality of the final report.

I would also like to thank the Department of Computer Science for the academic support and resources provided during the completion of this project.

Finally, I am grateful to my family and friends for their encouragement and support throughout this period.

Abstract

The rapid growth of academic publishing has made it increasingly difficult for researchers to identify relevant literature efficiently. Traditional academic retrieval systems often rely on lexical similarity, which performs well when related documents share similar vocabulary but is limited when conceptually related papers use different terminology. This project investigates whether semantic similarity techniques provide a more effective basis for academic recommendation.

The aim of the project was to compare lexical and semantic similarity measurement techniques within a controlled content-based academic recommendation system. An OpenAlex-derived dataset of 978 English-language Computer Science documents with abstracts was used to create a realistic but manageable experimental corpus. Two recommendation approaches were implemented within the same pipeline: a lexical baseline based on term frequency-inverse document frequency (TF-IDF) with cosine similarity, and a semantic approach based on transformer-based sentence embeddings.

The two methods were evaluated using ranking-based measures, including precision at k and normalised discounted cumulative gain at k , together with runtime analysis. The results showed that the semantic approach consistently achieved stronger recommendation quality than the lexical baseline, indicating better ranking of relevant academic documents. However, the semantic pipeline also imposed a substantially higher computational cost, particularly during embedding generation.

The project concludes that semantic similarity provides a meaningful advantage for academic recommendation in this setting, but that this improvement must be considered alongside practical constraints of efficiency and resource demand. Overall, the study contributes a controlled comparison of two important similarity paradigms and highlights the trade-off between recommendation effectiveness and computational feasibility in the design of academic recommendation systems.

Table of Contents

Declaration	2
Acknowledgements	3
Abstract	4
1 Introduction	11
1.1 Background of the study	11
1.2 Problem Statement	12
1.3 Aim and Objectives	12
1.4 Research Question	13
1.5 Significance of the Project	13
1.6 Project Contributions	14
1.7 Key Deliverables	14
1.8 Scope of the Project	15
1.9 Structure of the Report	15
1.10 Project Plan and Timeline	16
2 Literature Review	18
2.1 Introduction	18
2.2 Information Retrieval and Its Limitations in Academic Discovery	19
2.3 Scholarly Recommendation Systems	20
2.4 Lexical Similarity in Content-Based Recommendation	21
2.5 Semantic Similarity and Transformer-Based Sentence Embeddings	22
2.6 Open Scholarly Data and the Importance of Dataset Choice	24
2.7 Similarity and Evaluation Measures	24
2.8 Evaluation in Scholarly Recommendation	27

2.9 Comparative Synthesis of the Literature.....	28
2.10 Research Gap	30
3 Analysis and Design	31
3.1 Introduction	31
3.2 Overview of the Proposed System.....	31
3.3 Research Design	33
3.4 System Requirements	34
3.4.1 Functional requirements.....	34
3.4.2 Non-functional requirements	34
3.5 Dataset Selection and Justification	35
3.6 Recommendation Task Definition	37
3.7 Data Preparation Strategy.....	37
3.8 Document Representation Design	38
3.8.1 Lexical representation.....	38
3.8.2 Semantic representation.....	38
3.8.3 Comparative rationale.....	39
3.9 Similarity Computation and Ranking Logic	40
3.10 Evaluation Framework	41
3.11 Validity, Reliability, and Fairness	42
3.12 Feasibility and Design Constraints	42
3.13 Risk Assessment and Mitigation	43
4 Implementation	45
4.1 Introduction	45
4.2 Implementation Environment and Tools	45
4.3 Implementation Structure and Pipeline Realisation	46
4.4 Dataset Loading and Initial Processing	48
4.5 Text Preparation in Practice	49
4.6 Implementation of the Lexical Recommendation Branch	50
4.7 Implementation of the Semantic Recommendation Branch Using Sentence-BERT (SBERT) Embeddings.....	51

4.8	Similarity Computation and Recommendation Generation	53
4.9	Exporting Outputs for Testing and Evaluation	54
4.10	Computational Considerations During Implementation.....	55
4.11	Implementation Challenges and Resolutions	55
5	Testing.....	57
5.1	Introduction	57
5.2	Purpose and Scope of Testing	57
5.3	Testing Strategy	58
5.4	Testing Environment	59
5.5	Dataset and Input Validation Tests.....	59
5.6	Preprocessing and Representation Tests	60
5.7	Similarity and Ranking Tests.....	61
5.8	Output and Storage Verification	63
5.9	Runtime Analysis.....	63
5.10	Testing Outcomes and Interpretation	63
5.11	Limitations of the Testing Stage	64
6	Evaluation and Discussion.....	65
6.1	Introduction	65
6.2	Evaluation Purpose and Criteria.....	65
6.3	Evaluation Method.....	66
6.4	Quantitative Results	68
6.4.1	Runtime comparison	68
6.4.2	Precision at k.....	69
6.4.3	Normalised discounted cumulative gain at k.....	70
6.4.4	Dataset context.....	71
6.5	Discussion of Findings	71
6.6	Comparison with Existing Studies	72
6.7	Limitations of the Evaluation	73
6.8	Implications for the Project Objectives.....	74

7	Conclusion and Future Work	76
	7.1 Project Summary and Main Findings	76
	7.2 Key Contributions and Limitations	77
	7.3 Future Work and Final Remarks.....	77
8	References	78
9	Appendices.....	80
	Appendix A: Key Implementation Codes	80
	Sentence-BERT (SBERT) model.....	81
	Appendix B: Additional Outputs	81
	Lexical vs Semantic Embeddings Recommendation Output Example	81
	Runtime Comparison Output.....	82
	Meeting Logs.....	82
	Ethics Form.....	82

List of Figures

Figure 1.1 Gantt chart illustrating this Project Timeline (Week 1-12).....	17
Figure 2.1 Main approaches in scholarly recommendation systems (Author's own illustration). 21	
Figure 2.2 Conceptual comparison of lexical and semantic document similarity (Author's own illustration).....	23
Figure 3.1: Controlled recommendation pipeline for comparing lexical and semantic similarity approaches (Author's own illustration).	33
Figure 3.2: Comparison of lexical and semantic document representation within the project design (Author's own illustration).	40
Figure 4.1: Implemented recommendation pipeline showing the main stages of system execution (Author's own illustration).	48
Figure 4.2 Sample of the OpenAlex-derived dataset in tabular form showing document-level metadata.	48
Figure 4.3 TF-IDF vectorisation stage used to construct the lexical document representation ...	51
Figure 4.4 SBERT embedding generation using the all-MiniLM-L6-v2 model for semantic document representation.	53
Figure 4.5 Recommendation ranking functions for the lexical and semantic branches of the system.	54
Figure 5.1 Example top-k recommendation output comparing lexical (TF-IDF) and semantic (SBERT) approaches for a selected seed paper.....	62
Figure 6.1 Detailed evaluation results	67
Figure 6.2: Runtime comparison of lexical and semantic vector construction and recommendation generation stages (Project output).	68
Figure 6.3: Mean precision at k for lexical and semantic recommendation approaches under different relevance thresholds (Project output).	69
Figure 6.4: Mean normalised discounted cumulative gain at k for lexical and semantic recommendation approaches under different relevance thresholds (Project output).....	70

List of Tables

Table 2.1 Comparative summary of scholarly recommendation approaches.	29
Table 2.2 Comparative synthesis of lexical and semantic similarity approaches.	30
Table 3.1: Functional and non-functional requirements of the project.	35
Table 3.2: Dataset characteristics used in the project.	37
Table 3.3: Evaluation measures and their purpose in the project.	42
Table 3.4: Design-stage risks and mitigation strategies.	44
Table 5.1: Testing strategy adopted in the project.	59
Table 5.2: Sample functional test cases used in the project.	62
Table 6.1: Summary of observed performance patterns across lexical and semantic recommendation approaches.	71

List of Equations

Equation 1	25
Equation 2	26
Equation 3	26
Equation 4	27

1 Introduction

1.1 Background of the study

Academic publishing has expanded rapidly through digital dissemination, open-access infrastructures, and international research collaboration. Although this has improved access to knowledge, it has also made relevant literature harder to identify efficiently within large and growing document collections. Academic recommendation systems have therefore become increasingly important because they support discovery beyond simple keyword search and help researchers locate related work that might otherwise be overlooked (Zhang *et al.*, 2023).

Traditional academic retrieval systems are largely based on lexical information retrieval, where documents are represented through observable textual features and matched through word overlap or weighted term patterns. A widely used example is term frequency-inverse document frequency, which remains a useful baseline because it is efficient, interpretable, and effective when related documents use similar vocabulary. However, lexical methods depend heavily on surface wording and are therefore limited when conceptually related papers use different terminology (Zhang *et al.*, 2023). This vocabulary mismatch problem is especially significant in scholarly contexts, where similar ideas are often expressed differently across disciplines and subfields (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Recent advances in natural language processing have made semantic similarity methods more viable for recommendation tasks. Transformer-based sentence embedding models encode text as dense vectors that better capture contextual meaning, allowing conceptually related documents to be identified even when word overlap is limited. At the same time, prior research shows that embedding quality depends on model choice and application, and that semantic approaches often involve greater computational cost than lexical baselines (Li *et al.*, 2020; Zhang *et al.*, 2023).

This project builds on that context by using an OpenAlex-derived dataset to compare lexical and semantic similarity techniques within the same recommendation pipeline. The dataset contains 978 English-language Computer Science documents with abstracts, providing a controlled but realistic basis for experimentation. Preliminary results suggest that the semantic pipeline produces stronger ranking performance than the lexical baseline, but at a substantially higher

computational cost. The project therefore investigates not only recommendation effectiveness, but also the practical trade-off between performance and feasibility.

1.2 Problem Statement

The rapid growth of scholarly literature has made effective literature discovery increasingly difficult. Although search engines and digital libraries provide access to large collections of papers, traditional retrieval approaches still depend heavily on lexical overlap and are therefore better at finding documents that use similar wording than those that express similar ideas. This is a major limitation in academic contexts, where conceptually related work is often described with different terminology across subfields and publication communities (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Semantic methods have emerged as a promising alternative because they aim to represent documents in terms of contextual meaning rather than surface wording alone. However, existing research does not always provide a clear basis for deciding when semantic methods are preferable in practice, since many studies vary datasets, system architectures, and evaluation frameworks at the same time (Zhang *et al.*, 2023). In addition, embedding-based methods generally introduce greater computational cost and implementation complexity (Li *et al.*, 2020).

The core problem addressed by this project is therefore the lack of a clear, controlled comparison between lexical and transformer-based semantic similarity techniques for academic recommendation. Without such comparison, it remains difficult to justify which approach is more suitable when both recommendation quality and computational practicality are considered.

1.3 Aim and Objectives

The aim of this project is to conduct a comparative analysis of similarity measurement techniques for an academic recommendation system, with particular emphasis on lexical and semantic approaches.

To achieve this aim, the project addresses the following objectives:

1. To design a controlled recommendation pipeline for academic paper recommendation.
2. To implement a lexical similarity approach using TF-IDF and cosine similarity.

3. To implement a semantic similarity approach using transformer-based sentence embeddings, specifically through the Sentence-BERT (SBERT) model.
4. To evaluate both approaches using ranking-based evaluation metrics.
5. To compare their recommendation effectiveness under consistent experimental conditions.
6. To examine the computational and practical trade-offs associated with each approach.

1.4 Research Question

This project is guided by the following main research question:

How do lexical and semantic similarity measurement techniques compare in the context of an academic recommendation system?

To support this main question, the study is also informed by the following sub-questions:

- To what extent does semantic representation improve recommendation relevance over lexical representation?
- How do the two approaches differ in computational cost and practical feasibility?
- How do the findings of this project align with, or diverge from, recent work in scholarly recommendation research?

1.5 Significance of the Project

This project is significant because it addresses a practical problem in academic knowledge discovery while also contributing to a growing research area in scholarly recommendation. From a practical perspective, systems that recommend academic documents are increasingly important in helping researchers manage information overload and identify relevant literature more efficiently. From an academic perspective, the project contributes by isolating the role of similarity measurement within a controlled experimental design, rather than embedding the comparison within a broader hybrid system where multiple variables change at once. This improves analytical clarity and allows the comparative performance of lexical and semantic techniques to be interpreted more directly (Zhang *et al.*, 2023).

The project is also significant because it incorporates both methodological and implementation-level analysis. It does not assume that stronger recommendation quality automatically implies greater practical usefulness. Instead, it recognises that academic recommendation systems must balance ranking performance with issues such as efficiency, reproducibility, and deployment feasibility. By explicitly comparing effectiveness and computational cost, the project produces findings that are more useful for system design decisions than a purely accuracy-centred analysis would be.

1.6 Project Contributions

This project makes four main contributions.

First, it develops a controlled academic recommendation pipeline in which lexical and semantic similarity approaches can be compared under the same data conditions, recommendation logic, and evaluation framework.

Second, it applies this comparison to a realistic scholarly dataset derived from OpenAlex, thereby grounding the study in an open and reproducible data source suitable for academic recommendation research (Priem, Piwowar and Orr, 2022).

Third, it evaluates the two approaches not only in terms of ranking performance, but also in terms of computational cost, thereby producing a more balanced comparative analysis.

Fourth, it interprets the project findings in relation to recent scholarly recommendation literature, allowing the study to connect implementation outcomes with broader research debates in the field (Zhang *et al.*, 2023).

1.7 Key Deliverables

The key deliverables of the project are as follows:

- a curated dataset of academic documents derived from OpenAlex
- an implemented academic recommendation pipeline
- a lexical recommendation model based on TF-IDF and cosine similarity
- a semantic recommendation model based on transformer-based sentence embeddings
- a comparative evaluation using ranking-based metrics

- a final report presenting the design, implementation, testing, evaluation, and conclusions of the project

1.8 Scope of the Project

The scope of this project is deliberately focused. The project investigates **content-based academic recommendation** using textual metadata, primarily titles and abstracts, rather than collaborative filtering based on user interaction or citation-network-heavy hybrid recommendation. This is appropriate because the study aims to isolate the effect of similarity measurement rather than evaluate a full production-scale recommendation ecosystem.

The analysis is also limited to a curated subset of Computer Science-related records extracted from OpenAlex. While this improves feasibility and interpretability, it means that the findings should be understood as valid within the experimental context of the project rather than as universal claims about all disciplines or all large-scale industrial recommendation systems. This controlled scope is not a weakness in itself; rather, it is a methodological choice that supports a fair comparison between the techniques being investigated.

1.9 Structure of the Report

The remainder of the report is organised as follows.

Chapter 2 reviews the literature on information retrieval, scholarly recommendation systems, lexical similarity, semantic embeddings, and evaluation approaches, and identifies the research gap that motivates the project.

Chapter 3 presents the analysis and design of the project, including the methodological rationale, dataset strategy, recommendation task definition, and justification of the selected approaches.

Chapter 4 explains the implementation of the recommendation system, describing how the design was translated into a working computational pipeline.

Chapter 5 presents the testing procedures used to verify the correctness and reliability of the implemented system.

Chapter 6 evaluates and discusses the comparative performance of the lexical and semantic techniques, including ranking outcomes, computational trade-offs, and alignment with prior research.

Chapter 7 concludes the report by summarising the findings, identifying the main limitations, and proposing realistic directions for future work.

1.10 Project Plan and Timeline

The project was organised as a phased research and development process in order to ensure that the technical implementation remained aligned with the project aim, objectives, and submission requirements. Rather than treating the work as a single programming task, the project was planned across interconnected stages including literature review, problem refinement, dataset preparation, system design, implementation, testing, evaluation, and final report writing. This staged structure was important because the success of the project depended not only on building a functioning recommendation pipeline, but also on providing a well-justified comparative analysis of the similarity measurement techniques investigated.

The main project stages included an initial review of the literature on academic recommendation systems and similarity measurement, followed by dataset extraction and preparation using OpenAlex. This was then followed by the implementation of the lexical approach based on TF-IDF and cosine similarity, and the semantic approach based on transformer-based sentence embeddings. Once both approaches had been implemented, the later phases focused on testing system functionality, conducting comparative evaluation using ranking-based metrics, analysing runtime differences, and refining the written report.

The overall schedule for these activities is shown in **Figure 1.1**.

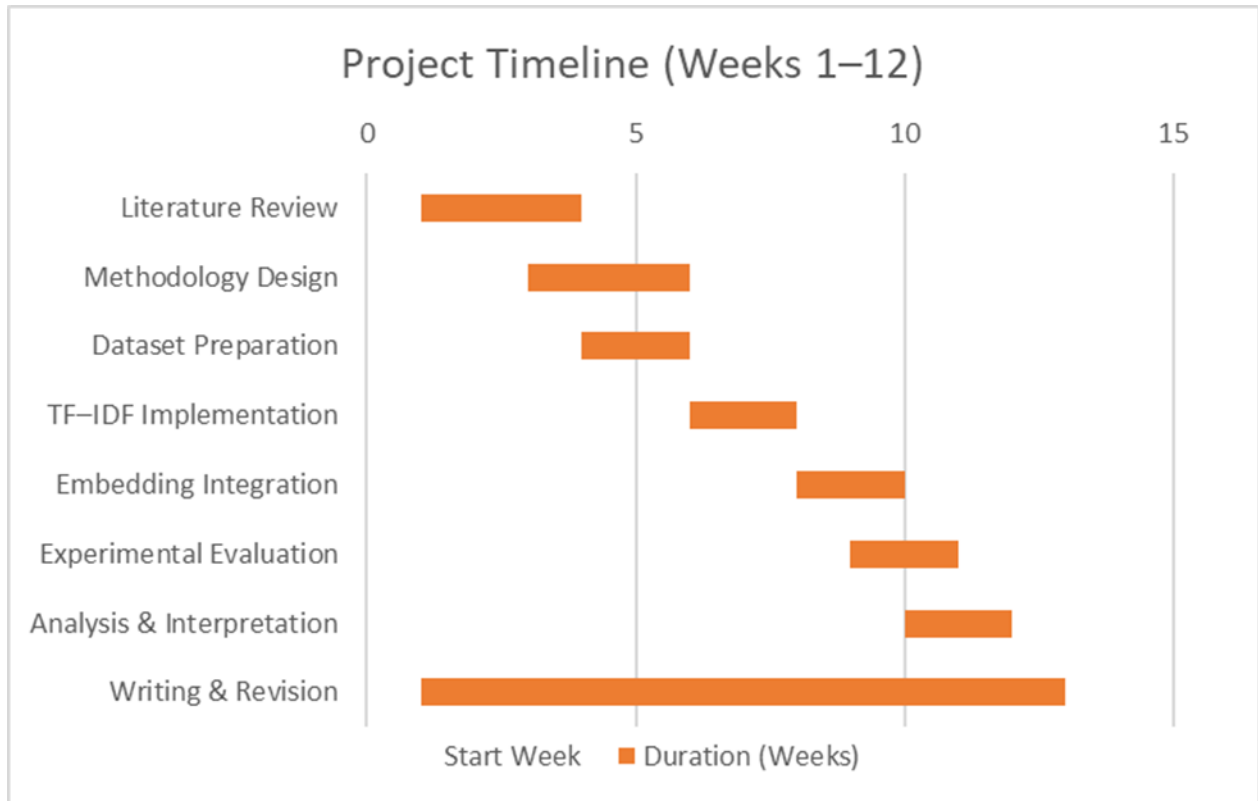


Figure 1.1 Gantt chart illustrating this Project Timeline (Week 1-12)

Overall, the timeline demonstrates that the project was managed in a deliberate and methodical manner. It also reflects an important principle underpinning the report as a whole: the project is not presented merely as a software artefact, but as a research-informed technical investigation in which planning, implementation, and evaluation are closely linked.

The following chapter now examines the literature in depth in order to develop the conceptual and empirical foundations for the methodological choices made later in the report.

2 Literature Review

2.1 Introduction

The previous chapter established that the project addresses the difficulty of identifying relevant academic literature within an increasingly large scholarly environment, and that this challenge is closely linked to how similarity between documents is measured. This chapter reviews the literature on academic recommendation systems, information retrieval, lexical and semantic similarity, and evaluation methods in order to justify the design of the project.

The review is intended not merely to summarise previous studies, but to identify the main conceptual debates, methodological patterns, and unresolved issues in the field. Recent surveys show that scholarly recommendation has become a distinct area of research, but one characterised by considerable variation in datasets, recommendation logic, and evaluation practices. While this diversity has encouraged innovation, it has also made direct comparison between methods more difficult, particularly when studies compare complete systems rather than specific components such as document representation or similarity computation (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

The literature also shows that scholarly recommendation differs from recommendation in commercial domains such as e-commerce or entertainment. Whereas commercial systems often rely heavily on user behaviour, scholarly recommendation frequently operates with limited interaction data and depends more directly on textual metadata such as titles, abstracts, and keywords. This makes content-based approaches especially important in academic contexts (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Within this setting, one of the key technical issues is the difference between lexical and semantic similarity. Lexical methods rely on visible word overlap, whereas semantic methods aim to capture contextual meaning more directly. Although the literature increasingly suggests that semantic approaches can improve recommendation quality, it also indicates that they are more computationally demanding and are not always evaluated under controlled conditions (Li *et al.*, 2020; Kreutz and Schenkel, 2022; Zhang *et al.*, 2023). This directly motivates the present project.

This chapter first reviews information retrieval and its limitations, then examines the main types of scholarly recommendation systems, before discussing lexical similarity, semantic similarity, evaluation methods, and the research gap addressed by the project.

2.2 Information Retrieval and Its Limitations in Academic Discovery

Information retrieval provides the foundation for many academic search and recommendation tools. It involves representing, indexing, and ranking documents in response to an information need. In academic contexts, this has traditionally meant retrieving papers whose textual content overlaps with a user query or a seed document. Such approaches remain widely used because they are efficient, interpretable, and scalable, especially when applied to structured metadata such as titles, abstracts, and keywords (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

One of the most established techniques in information retrieval is **term frequency-inverse document frequency (TF-IDF)**, which assigns greater weight to terms that are frequent in a document but less common across the corpus. This helps distinguish content-bearing terms from background vocabulary and explains why it remains a strong baseline in recommendation and retrieval research (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

However, lexical retrieval also has important limitations. The most significant is **vocabulary mismatch**, where conceptually related documents use different terminology and therefore appear less similar than they actually are. This problem is especially common in academic writing, where similar ideas may be expressed differently across subfields and disciplines (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023). Lexical methods are also relatively weak at handling synonymy, paraphrase, and context-sensitive meaning, since they treat words mainly as observable tokens rather than contextual units. As a result, they are often effective when terminology aligns closely, but less reliable when conceptual similarity extends beyond direct word overlap (Li *et al.*, 2020).

Even so, lexical methods remain valuable because they are efficient, transparent, and useful as baseline models. The key issue is therefore not whether they should be discarded, but whether their limitations are significant enough to justify more computationally expensive semantic

alternatives. For this project, that question is central, since the comparison depends on understanding both the strengths and the limitations of lexical retrieval.

2.3 Scholarly Recommendation Systems

Scholarly recommendation systems help users discover academic works relevant to a query, topic, document, or research interest. Their importance has grown alongside the expansion of academic publishing, which has made manual literature discovery increasingly difficult. Recent surveys show that the field includes content-based, collaborative, graph-based, citation-based, and hybrid approaches (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Scholarly recommendation differs from recommendation in domains such as e-commerce or entertainment because relevance is not simply a matter of predicting user preference. In academic contexts, recommendation supports knowledge discovery, methodological comparison, and citation expansion. As a result, scholarly systems place greater emphasis on document similarity, citation structure, topical relevance, and ranking quality than on conventional preference modelling (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Content-based recommendation is especially important in academic settings because it can operate without extensive user histories and can recommend newly added papers. By contrast, collaborative approaches rely on patterns of collective behaviour, such as downloads, readership, or citation relations, and may disadvantage new or sparsely connected papers.

Hybrid systems often improve performance by combining multiple signals, but they also make it harder to isolate the contribution of any single component. For this reason, a controlled content-based pipeline is appropriate for the present project, whose main aim is to compare similarity measurement techniques rather than model user behaviour or citation-network dynamics (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

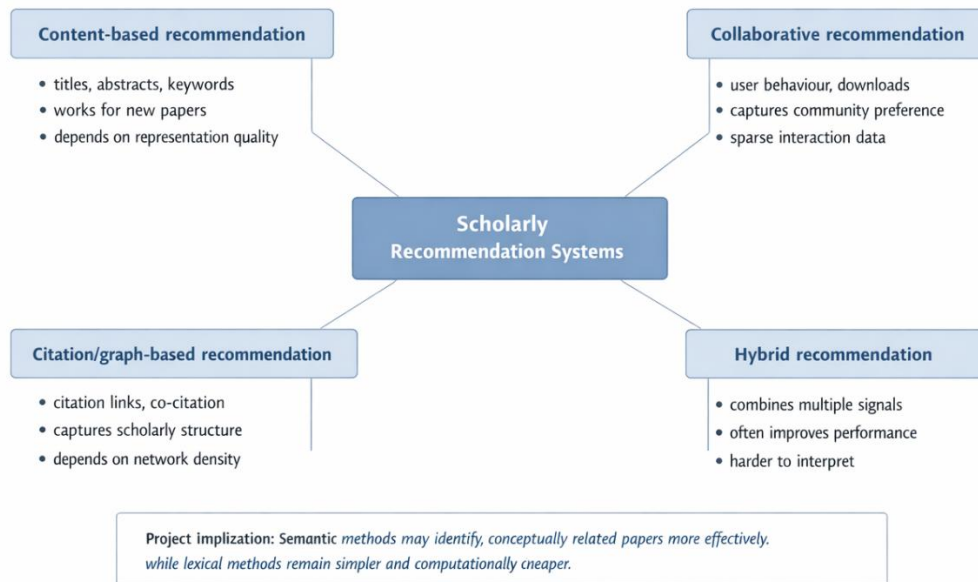


Figure 2.2: Main Approaches in Scholarly Recommendation Systems

Figure 2.1 Main approaches in scholarly recommendation systems (Author's own illustration).

Figure 2.1 should present a simple taxonomy showing **content-based**, **collaborative**, **graph/citation-based**, and **hybrid** recommendation approaches.

This focus on content-based recommendation is justified because it allows lexical and semantic similarity to be compared under the same data conditions, thereby aligning the literature review with the methodological logic of the project.

2.4 Lexical Similarity in Content-Based Recommendation

Lexical similarity refers to the extent to which two texts resemble each other through observable words or weighted term patterns. In recommendation systems, it is commonly implemented using term frequency-inverse document frequency with cosine similarity. This approach is mathematically simple, efficient, and often effective when documents share stable domain-specific vocabulary (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Its main limitation is that it depends heavily on surface wording. A term-weighting model cannot directly distinguish between semantic equivalence and lexical difference, nor can it easily account for context when the same word is used in different senses. This is especially problematic in scholarly literature, where similar ideas may be expressed through different technical or disciplinary language. In such cases, lexical similarity becomes only a partial indicator of relevance (Kreutz and Schenkel, 2022; Li *et al.*, 2020).

Even so, lexical methods remain valuable because they are efficient, transparent, and provide a strong baseline for comparative evaluation. The key issue is therefore not whether they should be abandoned, but whether their limitations are significant enough to justify more computationally expensive semantic alternatives. That question is central to the present project.

2.5 Semantic Similarity and Transformer-Based Sentence Embeddings

Semantic similarity aims to estimate relatedness in terms of meaning rather than visible word overlap. Instead of representing documents as sparse collections of weighted terms, semantic approaches encode them into dense vectors in which conceptual relationships are reflected geometrically. Documents that express related ideas may therefore be close in the vector space even when they use different wording (Li *et al.*, 2020).

A key development in this area is Sentence-BERT (SBERT), introduced by Reimers and Gurevych (2019). SBERT extends the BERT architecture to produce sentence embeddings that can be compared efficiently using cosine similarity. Unlike standard BERT, which is not optimised for direct similarity tasks, SBERT is specifically designed for semantic similarity and information retrieval, making it well suited to recommendation systems

Transformer models have strengthened semantic representation substantially, but the literature also shows that not all transformer outputs are equally suitable for similarity tasks. Li *et al.* (2020) demonstrate that sentence embeddings taken directly from pre-trained language models may perform poorly because they are not always optimised for semantic similarity. This highlights the need for careful model selection and evaluation.

In recommendation contexts, transformer-based sentence embeddings are attractive because they support efficient comparison once embeddings have been generated. Recent work has shown that such models can support recommendation through thematic matching of article metadata and that

model choice can materially affect outcomes (Colangelo *et al.*, 2025; Putra, Putri and Wiranatha, 2025). At the same time, semantic methods are typically more computationally demanding and less transparent than lexical baselines. The literature therefore suggests not a simple replacement of lexical methods, but a trade-off between semantic richness and computational feasibility (Colangelo *et al.*, 2025; Zhang *et al.*, 2023).

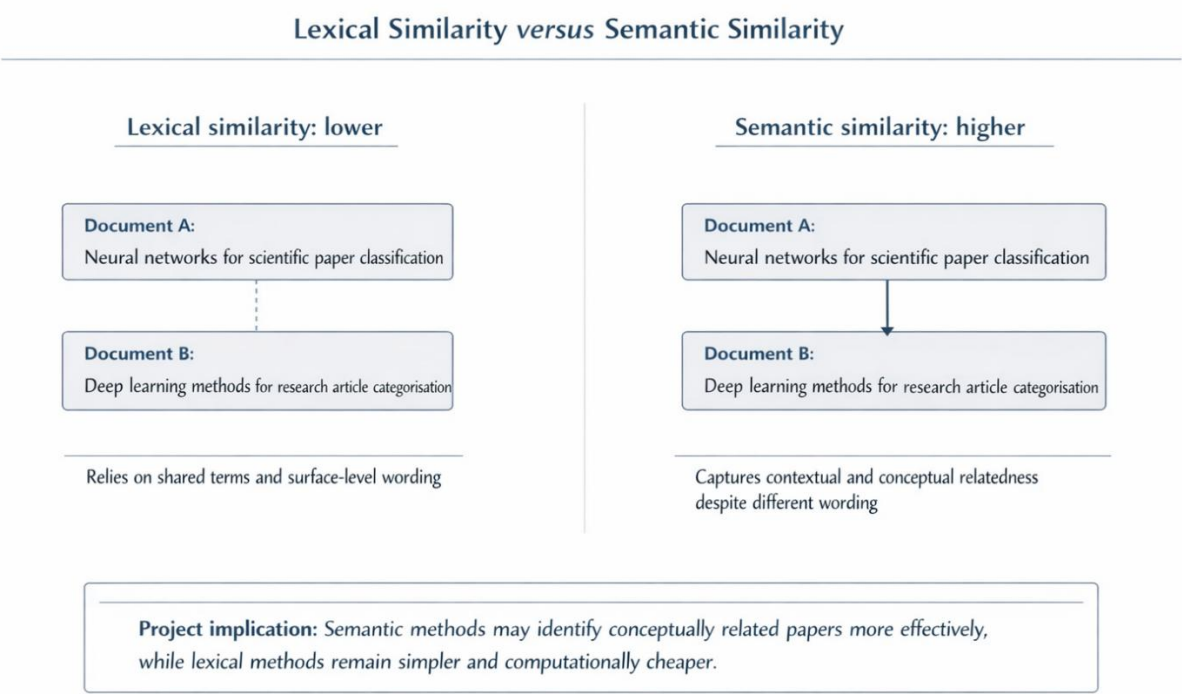


Figure 2.2 Conceptual comparison of lexical and semantic document similarity (Author’s own illustration).

Figure 2.2 should visually contrast two documents that use different wording but convey related meaning.

The literature does not support the claim that semantic methods are universally superior. A more defensible conclusion is that they are better suited to concept-level matching, but at greater

implementation and runtime cost. This is precisely why a controlled comparative project is justified.

2.6 Open Scholarly Data and the Importance of Dataset Choice

The literature also highlights the importance of dataset selection in scholarly recommendation research. Evaluation outcomes can vary substantially depending on the source, scale, domain, and structure of the data used, and recent reviews note that differences in corpora, relevance assumptions, and metadata quality often make cross-study comparison difficult (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

In this context, OpenAlex is especially relevant. Priem, Piwowar and Orr (2022) describe it as a fully open scholarly index designed to replace Microsoft Academic Graph, with coverage of works, authors, venues, institutions, and concepts. Its open access model and structured metadata make it highly suitable for reproducible recommendation experiments. For the present project, this is a significant advantage because it allows the recommendation pipeline to be grounded in realistic scholarly metadata without reliance on proprietary databases.

Dataset choice also shapes what can reasonably be claimed. A focused dataset based on open scholarly metadata can support controlled comparison between methods, but it cannot automatically be generalised to all disciplines or forms of scholarly discovery. The present project therefore uses a limited recommendation task and a bounded dataset in order to improve interpretability and reduce confounding variation.

2.7 Similarity and Evaluation Measures

To compare recommendation approaches meaningfully, it is necessary to define the mathematical measures used to represent document similarity and evaluate ranking quality. In this project, **cosine similarity** is used to compare document vectors for both the lexical and semantic approaches. Cosine similarity measures the angle between two vectors rather than their raw magnitude, making it appropriate for comparing both sparse text vectors and dense embedding vectors.

Cosine similarity

For two document vectors A and B, cosine similarity is defined as:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- $A \cdot B$ is the dot product of the two vectors
- $\|A\|$ is the magnitude of vector A
- $\|B\|$ is the magnitude of vector B

A value closer to 1 indicates stronger similarity, while a value closer to 0 indicates weaker similarity. In this project, cosine similarity is used consistently across both representation methods in order to isolate the effect of the document representation itself rather than the similarity function.

Precision at k

Precision at k measures the proportion of recommended documents within the top k results that are relevant:

$$\text{Precision@}k = \frac{\text{Number of relevant documents in top } k}{k}$$

Equation 1

Recall at k

Recall at k measures the proportion of all relevant documents that are successfully retrieved within the top k recommendations:

$$Recall@k = \frac{\text{Number of relevant documents retrieved}}{\text{Total number of relevant documents}}$$

Equation 2

Whereas precision focuses on the quality of the top-ranked results, recall indicates the extent to which the system captures the available relevant set.

Discounted cumulative gain at k

Discounted cumulative gain evaluates ranking quality by assigning greater importance to relevant items that appear earlier in the ranked list:

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i + 1)}$$

Equation 3

Where rel_i is the relevance score of the item at rank i . This position-sensitive weighting is appropriate in recommendation settings because earlier results are generally more valuable to users.

Normalised discounted cumulative gain at k

To support fair comparison across recommendation instances, discounted cumulative gain is normalised by the ideal discounted cumulative gain:

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$

Equation 4

Normalised discounted cumulative gain at k is especially useful in this project because it evaluates not only whether relevant documents are retrieved, but also how effectively they are ordered in the recommendation ranking.

Overall, these measures provide a balanced framework for comparing lexical and semantic recommendation approaches. Cosine similarity governs the generation of rankings, while precision at k , recall at k , and normalised discounted cumulative gain at k assess the quality of those rankings from different but complementary perspectives.

2.8 Evaluation in Scholarly Recommendation

Evaluation is one of the most challenging aspects of scholarly recommendation research because studies vary widely in relevance definitions, metrics, datasets, and protocols, making direct comparison difficult (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023). Even so, ranking-based metrics remain central. Measures such as **precision at k** , **recall at k** , and **normalised discounted cumulative gain at k** are widely used because recommendation systems are primarily judged by the quality of their ranked outputs. This is especially appropriate in academic contexts, where users typically focus on a small number of top recommendations rather than long lists of results (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Precision at k is useful because it measures the proportion of relevant items in the top results. Recall at k captures how much of the relevant set is retrieved, while normalised discounted cumulative gain at k adds positional sensitivity by rewarding methods that rank relevant papers earlier. This makes it especially suitable for the present project, which compares not only whether relevant documents are retrieved, but also how effectively they are ordered.

A further issue concerns the source of relevance judgements. Because large-scale scholarly recommendation studies often lack human-labelled relevance sets, they frequently rely on citation relations, metadata-based heuristics, or other proxy signals. Although imperfect, these approaches can still support meaningful comparison when applied consistently. The main

requirement is methodological transparency about how relevance is defined and what limitations follow from that choice (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

For the present project, these issues justify an evaluation framework centred on ranking metrics and controlled comparison rather than broad claims of universal superiority.

2.9 Comparative Synthesis of the Literature

Three main conclusions emerge from the literature. First, scholarly recommendation systems are highly heterogeneous. Content-based, collaborative, graph-based, and hybrid approaches all remain active, but this diversity makes it difficult to isolate the contribution of specific components such as document representation. For that reason, a focused content-based comparison remains methodologically valuable (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Second, lexical and semantic methods reflect different strengths rather than a simple old-versus-new divide. Lexical approaches remain efficient, interpretable, and useful in vocabulary-stable contexts, whereas semantic approaches are stronger at capturing conceptual similarity across varied phrasing but involve greater computational cost and lower interpretability. The literature therefore points to a trade-off rather than a universal winner (Li *et al.*, 2020; Colangelo *et al.*, 2025).

Third, many published comparisons are difficult to interpret because they vary multiple factors at once, including datasets, models, metadata sources, relevance definitions, and broader system architectures. This makes it hard to determine whether performance differences are truly caused by the representation method itself. This is the strongest justification for the present project’s design, which keeps the pipeline stable while varying the representation approach (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Approach	Main data source	Main strength	Main limitation	Relevance to this project
----------	------------------	---------------	-----------------	---------------------------

Content-based recommendation	Titles, abstracts, keywords, metadata	Works without rich user history; suitable for new papers	May overspecialise; sensitive to representation quality	Directly relevant
Collaborative recommendation	User behaviour, co-read patterns, ratings	Can capture community-level relevance	Suffers from sparsity and cold start	Not central
Citation/graph-based recommendation	Citation links, co-citation, bibliographic coupling	Captures scholarly network structure	Dependent on relational data quality and density	Contextually relevant
Hybrid recommendation	Combines text, graph, and/or behavioural data	Often improves performance	Harder to interpret and isolate component effects	Useful comparator, but outside main scope

Table 2.1 Comparative summary of scholarly recommendation approaches.

The role of Table 2.1 is to synthesise the broader field before the chapter narrows to the specific comparison of interest.

Criterion	Lexical similarity	Semantic similarity
Representation logic	Weighted surface terms	Dense contextual embeddings
Strength	Efficient, transparent, easy to reproduce	Better concept-level matching
Limitation	Vulnerable to vocabulary mismatch	Higher computational cost and lower interpretability

Best suited to	Stable domain vocabulary and baseline retrieval	Thematic recommendation where wording varies
Relevance to this project	Baseline model	Comparative target model

Table 2.2 Comparative synthesis of lexical and semantic similarity approaches.

Table 2.2 should be discussed explicitly in the text. Its purpose is to show that the comparison in this project is not arbitrary. Rather, it emerges directly from a recurring tension in the literature between **efficiency and interpretability** on one side, and **semantic richness and recommendation depth** on the other.

2.10 Research Gap

Although the literature on scholarly recommendation is substantial, a clear gap remains. Recent surveys show that many studies compare complete systems rather than isolating representation choices. As a result, there are many examples of lexical, semantic, graph-based, and hybrid recommendation, but fewer controlled studies that compare lexical and transformer-based semantic representations under the same dataset, task, similarity measure, and evaluation framework (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

This matters because it limits both theory and practice. Theoretically, it makes it harder to identify the specific contribution of document representation to recommendation quality. Practically, it leaves system designers uncertain whether the gains of semantic methods are sufficient to justify their added implementation and runtime cost in constrained environments.

The present project addresses this gap by using an open scholarly dataset, a consistent content-based recommendation pipeline, a lexical baseline based on term frequency-inverse document frequency, and a semantic approach based on transformer-based sentence embeddings. Its contribution lies not in proposing a complex hybrid system, but in providing a clearer comparison between two important similarity paradigms within academic recommendation.

3 Analysis and Design

3.1 Introduction

Chapter 2 showed that scholarly recommendation systems are methodologically diverse and that direct comparison between lexical and semantic approaches is often weakened by differences in datasets, architectures, and evaluation strategies. In response, this chapter presents the analysis and design of the project and explains how the recommendation system was planned before implementation.

The project adopts a controlled comparative design in which the main variable is the document representation method. The lexical branch uses term frequency-inverse document frequency, while the semantic branch uses transformer-based sentence embeddings. Other core elements of the pipeline, including the dataset, recommendation task, similarity computation, and evaluation framework, are kept as stable as possible. This is important because recent reviews note that many scholarly recommendation studies vary multiple components at once, making it difficult to isolate the specific contribution of a similarity method (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Therefore, this chapter outlines the design logic of the system, covering the overall system concept, research design, requirements, dataset selection, task definition, data preparation, document representation, similarity computation, evaluation framework, and issues of validity, feasibility, and risk.

3.2 Overview of the Proposed System

The proposed system is a **content-based academic recommendation system** designed to generate document-to-document recommendations. Given a seed paper, the system retrieves and ranks the most similar papers from the dataset according to their textual similarity. This design is appropriate because the project is concerned with comparing document similarity techniques rather than modelling user behaviour, collaborative interactions, or citation graph propagation.

At a high level, the system consists of five stages: dataset acquisition and filtering, text preparation, document representation, similarity computation and ranking, and evaluation. The

central design feature is that the overall workflow remains constant while the representation method changes. In the lexical branch, documents are converted into sparse weighted term vectors using term frequency-inverse document frequency. In the semantic branch, documents are encoded into dense contextual vectors using a transformer-based sentence embedding model. Both branches then apply cosine similarity to produce ranked recommendation lists. This use of cosine similarity across both branches is methodologically important because cosine similarity is standard for comparing both sparse and dense vector spaces, and its consistent use helps isolate the effect of representation rather than introducing further variation through different similarity functions (scikit-learn, 2025).

This deliberately controlled architecture reflects the analytical goal of the project. The system is not intended to be a full production-scale academic recommender incorporating hybrid signals, user histories, or graph neural models. Instead, it is designed to answer a bounded but important question in a clear way: whether and how lexical and semantic representations differ in recommendation effectiveness and computational cost when applied to the same scholarly recommendation task.

Figure 3.1 summarises the controlled recommendation pipeline designed for the project, in which the same dataset, ranking logic, and evaluation framework are retained while the representation method changes (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

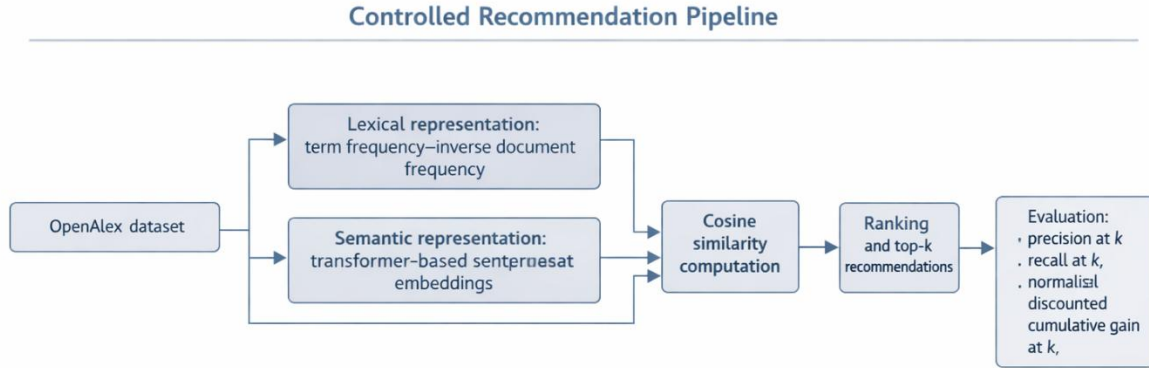


Figure 3.1: Controlled recommendation pipeline for comparing lexical and semantic similarity approaches (Author’s own illustration).

3.3 Research Design

The project adopts a **comparative experimental research design**. Comparative experimentation is appropriate when the purpose of a study is to examine how two or more alternative methods behave under equivalent conditions. In this project, the comparison focuses specifically on two document representation paradigms: a lexical approach based on term frequency-inverse document frequency and a semantic approach based on transformer-based sentence embeddings.

A controlled design is essential because the literature shows that recommendation results are often influenced by multiple interacting factors, including the dataset used, the richness of metadata, the ranking function, and the relevance assumptions adopted during evaluation (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023). To reduce such confounding influences, the project fixes the dataset, the recommendation task, the similarity function, and the evaluation metrics. The main variable that changes is therefore the representation technique.

This design also reflects the principle of **methodological fairness**. Fair comparison does not necessarily mean identical processing at every stage. Rather, it means that both methods should be implemented competently, under conditions suitable to their design, while being evaluated within the same broader framework. This is especially relevant to preprocessing, since lexical methods and transformer-based embedding methods do not rely on text in the same way.

3.4 System Requirements

The project required both functional and non-functional design requirements.

3.4.1 Functional requirements

The system needed to:

- ingest a filtered academic dataset
- process textual document metadata
- generate lexical and semantic document representations
- compute similarity scores between a seed paper and candidate papers
- rank papers according to similarity
- return the top-k recommendations
- evaluate recommendation performance using ranking metrics
- compare runtime behaviour between methods

3.4.2 Non-functional requirements

The system also needed to satisfy several non-functional requirements:

- **reproducibility**, so that the workflow could be rerun consistently
- **modularity**, so that representation methods could be compared without redesigning the full system
- **computational feasibility**, so that the project remained practical within undergraduate resource limits
- **consistency**, so that comparative evaluation remained fair

- **interpretability**, particularly for the lexical baseline and evaluation logic

These requirements reflect the fact that the project is both technical and analytical. The system is not only expected to function correctly but also to support a defensible comparison between methods.

Requirement type	Requirement	Justification
Functional	Generate recommendations from a seed paper	Core project task
Functional	Support lexical and semantic representations	Required for direct comparison
Functional	Rank papers using similarity scores	Necessary for top-k recommendation
Functional	Compute evaluation metrics	Required for comparative assessment
Non-functional	Reproducibility	Supports transparent experimentation
Non-functional	Modularity	Allows branch comparison without redesign
Non-functional	Computational feasibility	Necessary within project constraints
Non-functional	Consistency	Ensures fair evaluation conditions

Table 3.1: Functional and non-functional requirements of the project.

3.5 Dataset Selection and Justification

The dataset used in the project was derived from **OpenAlex**, a fully open index of scholarly works, authors, venues, institutions, and concepts (Priem, Piwowar and Orr, 2022). OpenAlex is

particularly suitable for this project because it provides structured scholarly metadata in an open and reproducible form, making it well suited to academic recommendation research.

The extracted dataset was filtered to include documents with abstracts, English-language publications, papers from 2015 onwards, and works associated with the Computer Science concept area. According to the project’s extraction metadata, the query used the OpenAlex works endpoint with the filters `has_abstract:true`, `language:en`, `from_publication_date:2015-01-01`, and `concepts.id:C41008148`, together with a cited-by-count filter and a fixed seed of 42. The extraction collected 1,000 records across five pages at 200 records per page, and the final usable project dataset contained **978 documents**.

The dataset characteristics support its suitability for controlled experimentation. The corpus has an **average abstract length of 168.23 words**, a **median abstract length of 162 words**, an **average citation count of 938.98**, and a **median citation count of 935**. These values indicate that the documents contain sufficient textual content for both lexical and semantic comparison while remaining computationally manageable for repeated experimentation.

The choice of a moderate-sized filtered corpus is deliberate. A much smaller dataset would weaken the credibility of the evaluation, whereas a much larger dataset could shift the project toward large-scale engineering rather than controlled comparative analysis. The selected OpenAlex subset therefore provides a reasonable balance between realism, reproducibility, and feasibility.

Characteristic	Value
Data source	OpenAlex
Domain focus	Computer Science
Language	English
Publication period	2015 onwards
Number of documents	978

Average abstract length	168.23 words
Median abstract length	162 words
Average citation count	938.98
Median citation count	935
Extraction seed	42

Table 3.2: Dataset characteristics used in the project.

3.6 Recommendation Task Definition

The project implements a **document-to-document recommendation task**. In this setting, a selected paper acts as the seed document, and the system returns the most similar papers from the remaining corpus. This is an appropriate task design because it reflects a realistic academic workflow in which a researcher begins with one relevant paper and seeks related literature.

This task was chosen for three reasons. First, it aligns naturally with content-based recommendation because the system relies on textual metadata rather than user interaction histories. Second, it supports direct comparison between lexical and semantic similarity approaches without introducing collaborative or graph-based confounds. Third, it fits the broader project objective of comparing similarity measurement techniques rather than building a personalised recommender.

The recommendation process is straightforward: a seed document is represented, compared with the remaining documents, assigned similarity scores, and then used to produce a ranked list of top-k candidate papers. This ranking-based formulation also aligns closely with the evaluation metrics used later in the project, particularly precision at k, recall at k, and normalised discounted cumulative gain at k.

3.7 Data Preparation Strategy

Data preparation is a crucial design stage because the quality of document representation depends heavily on the structure of the input text. In this project, preprocessing was designed according to the needs of each representation method rather than enforced identically across both branches.

For the lexical branch, preprocessing aimed to reduce noise and improve term weighting. This involved standard text cleaning choices such as lowercasing, punctuation removal, and stop-word handling, all of which are consistent with the use of term frequency-inverse document frequency vectorisation. The scikit-learn vectoriser is widely used for this purpose and supports document-to-term matrix construction with inverse-document-frequency weighting and vector normalisation (scikit-learn, 2025).

For the semantic branch, preprocessing was intentionally lighter. Transformer-based sentence embedding models operate on more natural text sequences and derive meaning from contextual patterns. Over-aggressive cleaning can therefore remove useful information and weaken the model’s representational strengths. This distinction is methodologically justified: fair comparison requires appropriate processing for each method, not artificial uniformity that disadvantages one branch.

3.8 Document Representation Design

Document representation is the central design variable in the project because the research question concerns the comparative behaviour of lexical and semantic similarity methods.

3.8.1 Lexical representation

In the lexical branch, each paper is represented using **term frequency-inverse document frequency**. This representation converts each document into a sparse vector in which each dimension corresponds to a term and each value reflects that term’s weighted importance within the corpus. The method remains a strong and widely used baseline in text retrieval because it is efficient, interpretable, and easy to implement at moderate scale (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

The lexical representation was chosen because it provides a transparent baseline against which the semantic method can be assessed. Its inclusion is not merely conventional; it is analytically important, since it allows the project to determine whether the additional complexity of semantic embeddings yields meaningful recommendation gains.

3.8.2 Semantic representation

In the semantic branch, each paper is represented using **transformer-based sentence embeddings**. Sentence-BERT was introduced specifically to generate semantically meaningful

sentence embeddings that can be compared efficiently using cosine similarity, making it especially suitable for semantic similarity and retrieval tasks (Reimers and Gurevych, 2019). Subsequent work also shows that sentence embedding quality is not automatic across all pre-trained language models, which makes careful model use and evaluation important (Li *et al.*, 2020).

This representation is suitable for the project because it allows the system to compare documents on the basis of contextual meaning rather than visible term overlap alone. However, the project also recognises that semantic embeddings typically involve greater representation cost than lexical vectorisation. That practical trade-off is part of the comparison rather than a side issue.

3.8.3 Comparative rationale

The contrast between the two representation methods is deliberate. The lexical branch reflects a sparse, surface-oriented approach to similarity, while the semantic branch reflects a dense, context-oriented approach. Comparing them within the same pipeline makes it possible to assess whether semantic representation improves recommendation quality sufficiently to justify its higher computational cost in a realistic academic recommendation scenario.

Figure 3.2 shows the conceptual difference between the two representation methods used in the project, namely sparse weighted lexical vectors and dense contextual semantic embeddings (Reimers and Gurevych, 2019; Li *et al.*, 2020).

Lexical and Semantic Document Representation

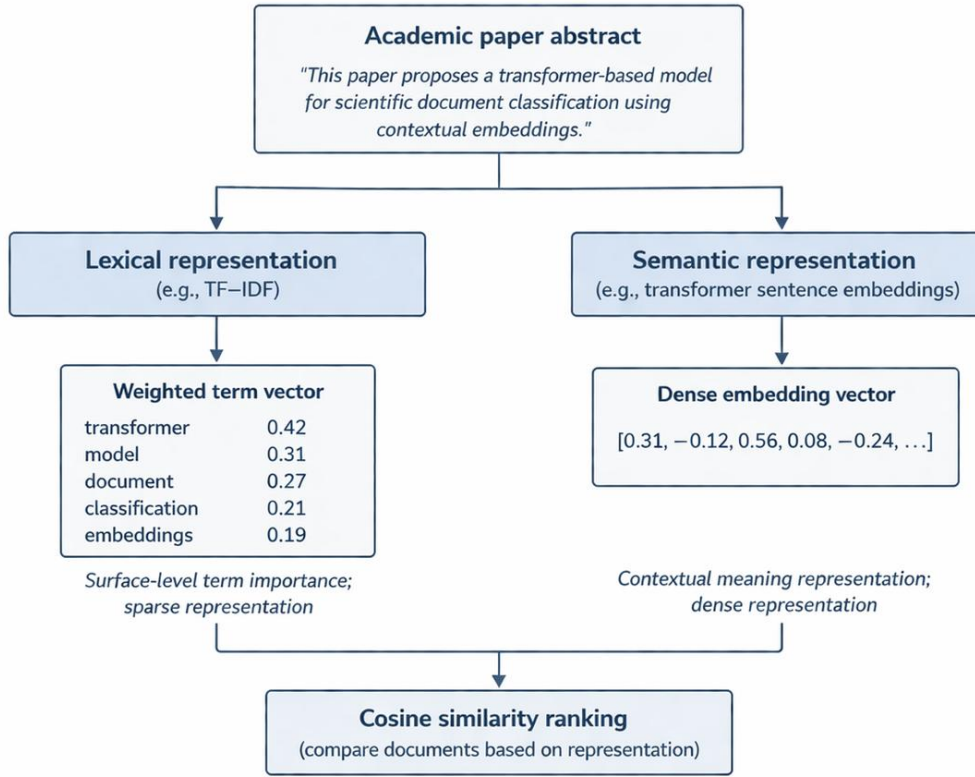


Figure 3.2: Comparison of lexical and semantic document representation within the project design (Author’s own illustration).

3.9 Similarity Computation and Ranking Logic

After representation, similarity scores are computed using **cosine similarity** in both branches. Cosine similarity measures the cosine of the angle between two vectors and is commonly used for both sparse text vectors and dense embeddings. In scikit-learn, cosine similarity is defined as the normalised dot product between samples, which makes it appropriate for comparing vector direction rather than raw magnitude (scikit-learn, 2025).

Using the same similarity function in both branches is methodologically important. If the project had changed both the representation method and the similarity function, then any difference in

recommendation performance could not confidently be attributed to representation alone. By holding cosine similarity constant, the project preserves a clearer comparative focus.

Once similarity scores are computed between the seed document and the remaining documents, the candidate papers are ranked in descending order. The top-k highest scoring papers are then returned as recommendations. This ranking-based design supports both recommendation generation and formal evaluation.

3.10 Evaluation Framework

The evaluation framework was designed to assess both **recommendation quality** and **computational feasibility**. In academic recommendation systems, ranking quality is especially important because users typically focus on the top portion of the recommendation list rather than inspecting all possible candidates. Recent surveys of scholarly recommendation systems identify **precision**, **recall**, and **normalised discounted cumulative gain** as common and important metrics in offline evaluation. A recent review of scholarly publication recommender systems likewise notes that precision, recall, mean reciprocal rank, mean average precision, and normalised discounted cumulative gain remain among the most common metrics used in this area (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023; Khadka and Sthapit, 2025).

Accordingly, the project uses:

- **precision at k**, to assess the concentration of relevant papers in the top recommendations
- **recall at k**, to assess how much of the relevant set is retrieved
- **normalised discounted cumulative gain at k**, to assess ranking quality with position sensitivity
- **runtime**, to compare the computational cost of the two methods

Including runtime as part of the framework is essential. A semantic method may achieve better ranking performance, but if it requires substantially greater time or resources, that has practical implications for system design. Since one of the project objectives is to examine trade-offs as well as effectiveness, runtime forms part of the evaluation rather than sitting outside it.

Measure	What it assesses	Why it is included
---------	------------------	--------------------

Precision at k	Relevance density in top-ranked results	Reflects usefulness of top recommendations
Recall at k	Coverage of relevant documents	Indicates retrieval breadth
Normalised discounted cumulative gain at k	Ranking quality with position weighting	Rewards early placement of relevant items
Runtime	Computational cost	Supports feasibility comparison

Table 3.3: Evaluation measures and their purpose in the project.

3.11 Validity, Reliability, and Fairness

Internal validity is strengthened by holding constant the dataset, recommendation task, similarity function, and evaluation metrics. This allows performance differences to be interpreted more confidently as differences between lexical and semantic representation rather than between unrelated system components.

External validity is more limited. Because the project uses a curated Computer Science subset of OpenAlex and a content-based recommendation task based mainly on textual metadata, the findings are most applicable to similar scholarly recommendation contexts rather than to all disciplines or production-scale systems. This is acceptable because the aim is controlled comparison rather than universal generalisation.

Reliability is supported by explicit filtering criteria, a fixed extraction seed, and a structured workflow. Reproducibility is further strengthened by the use of OpenAlex as an open data source and by reliance on widely used Python tools and established models (Priem, Piwowar and Orr, 2022; scikit-learn, 2025). Fairness is maintained by using the same broader architecture and evaluation conditions for both methods while allowing preprocessing appropriate to each representation type.

3.12 Feasibility and Design Constraints

The project was designed under realistic temporal and computational constraints. The lexical branch is relatively lightweight, whereas the semantic branch requires more processing during

representation generation, even though similarity comparison after embedding remains efficient (Reimers and Gurevych, 2019).

These constraints influenced the overall design. The dataset had to be large enough to produce meaningful ranking behaviour but small enough to remain tractable in an environment such as Google Colab. The project also focuses on titles and abstracts rather than full text, which improves feasibility while retaining the main semantic content needed for comparison.

Feasibility is therefore part of the methodology rather than separate from it. A design that can be implemented and evaluated consistently is stronger than one that is more ambitious in theory but impractical in execution.

3.13 Risk Assessment and Mitigation

Several design-stage risks were identified. Incomplete metadata could weaken document representation, so the dataset was filtered to include papers with abstracts and English-language content. Semantic processing could be substantially slower than lexical processing, so a moderate dataset size and a simple content-based architecture were used. Evaluation could also depend on proxy relevance rather than fully human-labelled judgements, which was addressed by using multiple ranking metrics and interpreting the results comparatively rather than as an absolute claim of universal relevance. A further risk was that the project might become too architecturally broad, making the comparison harder to interpret. This was mitigated by keeping the system focused on content-based similarity comparison.

Risk	Potential impact	Mitigation strategy
Incomplete metadata	Weak representation quality	Filter to records with abstracts
High semantic processing cost	Slower experimentation	Use moderate dataset size
Proxy relevance assumptions	Evaluation bias	Use multiple ranking metrics and state limitations

Excessive system complexity	Reduced interpretability	Maintain a controlled content-based design
-----------------------------	--------------------------	--

Table 3.4: Design-stage risks and mitigation strategies.

These design decisions provide the foundation for the implementation described in Chapter 4.

4 Implementation

4.1 Introduction

Chapter 3 outlined the analytical and methodological design of the project. This chapter explains how that design was translated into an operational system. It focuses on practical execution, including dataset handling, pipeline construction, implementation of the lexical and semantic branches, and preparation of outputs for later testing and evaluation.

Implementation is treated here as the practical realisation of the comparative framework developed earlier, rather than as a purely technical exercise. The aim is therefore not only to describe what was built, but also to show how the implementation remained aligned with the project objectives and research problem.

The system was developed in Python within a notebook-based environment using standard scientific and natural language processing libraries. The workflow was organised as a modular pipeline so that each stage of the recommendation process could be implemented and checked separately. This was especially important because the project compares two document representation strategies while keeping the broader recommendation logic constant.

This chapter is organised as follows. First, the implementation environment and system structure is described, then explains dataset preparation, lexical and semantic implementation, ranking, output generation, and the main challenges encountered during development.

4.2 Implementation Environment and Tools

The system was implemented in Python using a notebook-based development environment. The notebook was used to organise the workflow into clear stages, including dataset loading, preprocessing, document representation, similarity computation, and result generation. This structure made it easier to track development progress and review intermediate outputs during implementation.

The implementation relied primarily on the following tools and libraries:

- **Pandas**, for data loading, manipulation, and tabular processing
- **NumPy**, for numerical operations and array handling

- **scikit-learn**, for term frequency-inverse document frequency vectorisation and cosine similarity computation
- **sentence-transformers**, for generating transformer-based sentence embeddings
- **Matplotlib** and related plotting tools, for producing visual summaries of runtime and evaluation outputs
- **Google Colab**, as the practical environment for development and execution

These tools were selected because they are widely used, reliable, and appropriate for text-based recommendation experiments. The lexical branch required efficient sparse vectorisation and standard similarity computation, both of which are well supported by scikit-learn. The semantic branch required access to transformer-based embedding models, for which the sentence-transformers library was particularly suitable because it enables the generation of semantically meaningful sentence embeddings for similarity tasks (Reimers and Gurevych, 2019).

The notebook environment was therefore used mainly as a practical development and execution space for the project, allowing the implementation steps to be run, checked, and refined in a structured manner. This was especially useful because the project required repeated comparison between a lightweight lexical pipeline and a more computationally demanding semantic pipeline within the same working environment.

4.3 Implementation Structure and Pipeline Realisation

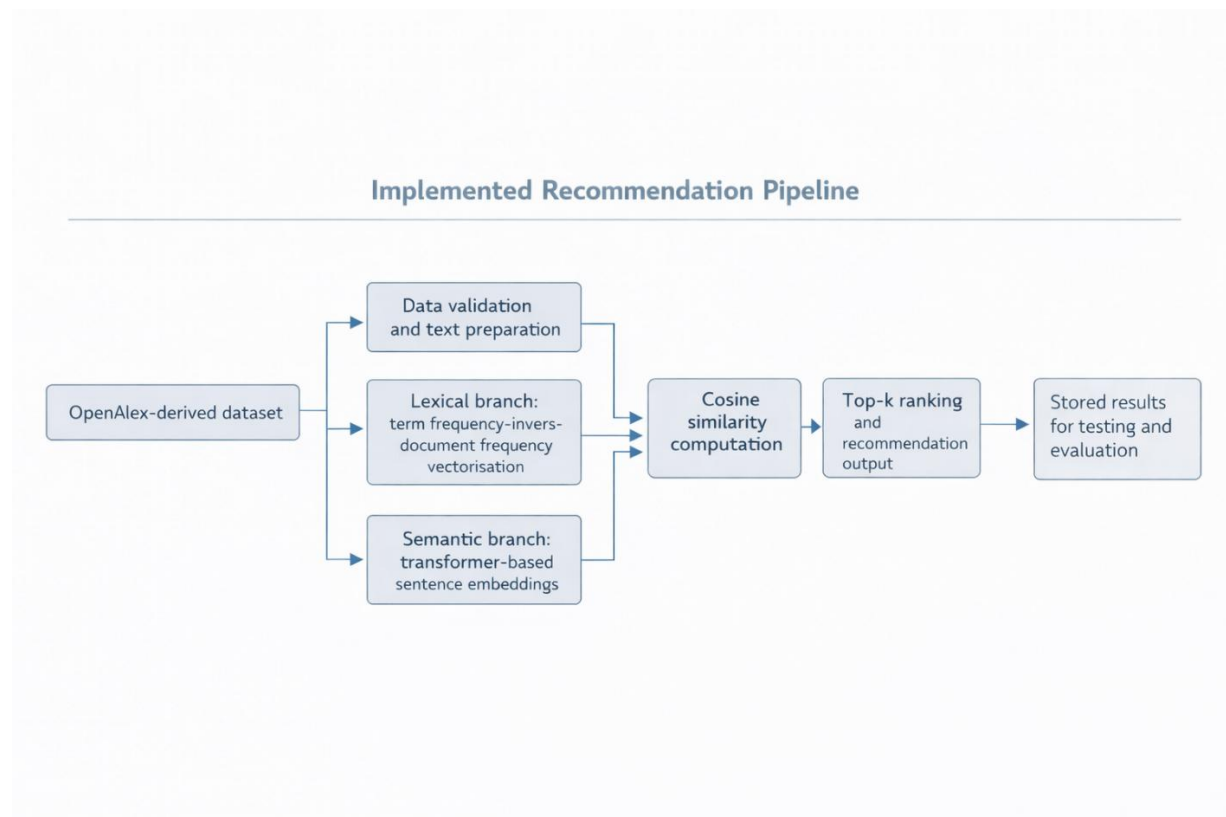
The implementation followed the pipeline designed in Chapter 3 and was organised into a sequence of interconnected stages. These stages were:

1. loading and inspecting the dataset
2. filtering and preparing the textual content
3. constructing lexical document representations
4. constructing semantic document representations
5. computing pairwise similarity scores
6. generating ranked recommendations

7. exporting quantitative outputs for testing and evaluation

Each stage was implemented as a distinct part of the workflow so that errors could be identified more easily and so that the lexical and semantic branches could be compared without altering the overall logic of the system. In practical terms, this meant that the same input corpus, recommendation task, and ranking structure were maintained across both branches, while only the representation stage differed.

This modular arrangement also strengthened the analytical value of the implementation. Rather than building one opaque end-to-end process, the system was realised as a structured workflow in which data processing, representation, ranking, and evaluation preparation could be examined individually. This was beneficial from both a technical and academic perspective. From a technical perspective, modularity improved maintainability and debugging. From an academic perspective, it improved transparency, making it easier to explain how the implementation reflected the comparative logic of the project.



	id	title	year	citations	concepts	abstract	abstract_length_words
0	https://openalex.org/W1408981388	Machine Learning methods for Quantitative Radi...	2015	999	['id': 'https://openalex.org/C2778559731', 'w...	Radiomics extracts and mines large number of m...	211
1	https://openalex.org/W2317136217	IMAGE ARTIFACTS IN OPTICAL COHERENCE TOMOGRAPH...	2015	999	['id': 'https://openalex.org/C31972630', 'wik...	Image artifacts are common and can lead to inc...	44
2	https://openalex.org/W2606037183	A Bioinspired Mineral Hydrogel as a Self-Heala...	2017	999	['id': 'https://openalex.org/C192562407', 'wi...	In the past two decades, artificial skin-like ...	129
3	https://openalex.org/W2918612296	Tailoring the photoluminescence of atomically ...	2019	999	['id': 'https://openalex.org/C2778402822', 'w...	Due to their atomically precise structures and...	209
4	https://openalex.org/W3103891289	Chatbots: History, technology, and applications	2020	999	['id': 'https://openalex.org/C41008148', 'wik...	This literature review presents the History, T...	147

Figure 4.1: Implemented recommendation pipeline showing the main stages of system execution (Author's own illustration).

Figure 4.1 represents how the OpenAlex-derived dataset was transformed into lexical and semantic representations before similarity computation, ranking, and output generation. Its inclusion is important because it shows continuity between the design decisions presented in Chapter 3 and their practical realisation in the implemented system.

4.4 Dataset Loading and Initial Processing

Implementation began with the loading of the project dataset derived from OpenAlex. As established in Chapter 3, the filtered corpus contained 978 documents with textual metadata suitable for recommendation analysis. The dataset was imported into the notebook environment in structured tabular form, allowing each document to be treated as an individual record with associated fields such as title, abstract, and citation-related metadata. An example of this structured dataset is shown in Figure 4.2, illustrating how each document is represented with its corresponding attributes.

Figure 4.2 Sample of the OpenAlex-derived dataset in tabular form showing document-level metadata.

The use of a tabular format was appropriate because it enabled efficient indexing, filtering, and alignment between document identifiers and their generated vector representations.

At this stage, the main implementation priority was **data integrity**. Before any representation or similarity computation could be performed, the dataset had to be checked for missing or inconsistent values, particularly in the abstract field, which formed the main textual input to the recommendation system. The earlier filtering process had already excluded papers without abstracts, but practical implementation still required verification that the textual column used for

representation was consistently formatted and free from null values that could interrupt the vectorisation process.

The implementation also involved establishing a stable correspondence between each document and its position in the dataset. This was important because later stages of the pipeline, particularly similarity ranking and result interpretation, required reliable mapping between vector indices and document identities. Without this consistency, the ranking outputs would be difficult to interpret and potentially unreliable for evaluation purposes.

The initial data loading stage therefore served not only as a technical input step but also as a foundation for the credibility of the later implementation. In recommendation systems, weak data handling at the beginning of the workflow can compromise the entire downstream process. For this reason, careful dataset inspection and validation formed an essential part of the implementation.

4.5 Text Preparation in Practice

Although the broad preprocessing strategy was justified in Chapter 3, implementation required those principles to be converted into practical processing steps.

For the **lexical branch**, the text was normalised in order to improve the quality of term weighting. This involved lowercasing, removal of punctuation and extraneous symbols, and stop-word handling during vectorisation. These choices are consistent with standard preprocessing practice for sparse lexical representations because they reduce noise and help ensure that highly frequent but semantically weak tokens do not dominate the document vectors.

For the **semantic branch**, preprocessing was deliberately more conservative. The textual content was kept closer to its natural sentence form so that the transformer-based sentence embedding model could exploit contextual information. This distinction between the two branches was a direct implementation of the fairness principle described earlier: the goal was not to force identical text cleaning across both methods, but to ensure that each method operated under conditions appropriate to its representational logic.

In practical terms, this meant that implementation did not consist of a single universal cleaning process applied to all text. Instead, preprocessing was tied to the requirements of each pipeline. This was a more defensible implementation choice because it avoided weakening the semantic

branch through unnecessary stripping of contextual structure, while still allowing the lexical branch to benefit from classical noise reduction.

A further implementation consideration was consistency. Although preprocessing differed between branches, it was applied consistently within each branch across all documents. This ensured that the later similarity comparisons were methodologically coherent and not distorted by inconsistent text handling.

4.6 Implementation of the Lexical Recommendation Branch

The lexical branch was implemented using **term frequency-inverse document frequency vectorisation** followed by cosine similarity. In practical terms, the textual metadata of each document, primarily the abstract, was converted into a sparse vector representation in which each term received a weighted value based on its importance within the document and rarity across the corpus. This process transformed the document collection into a document-term matrix suitable for efficient similarity computation.

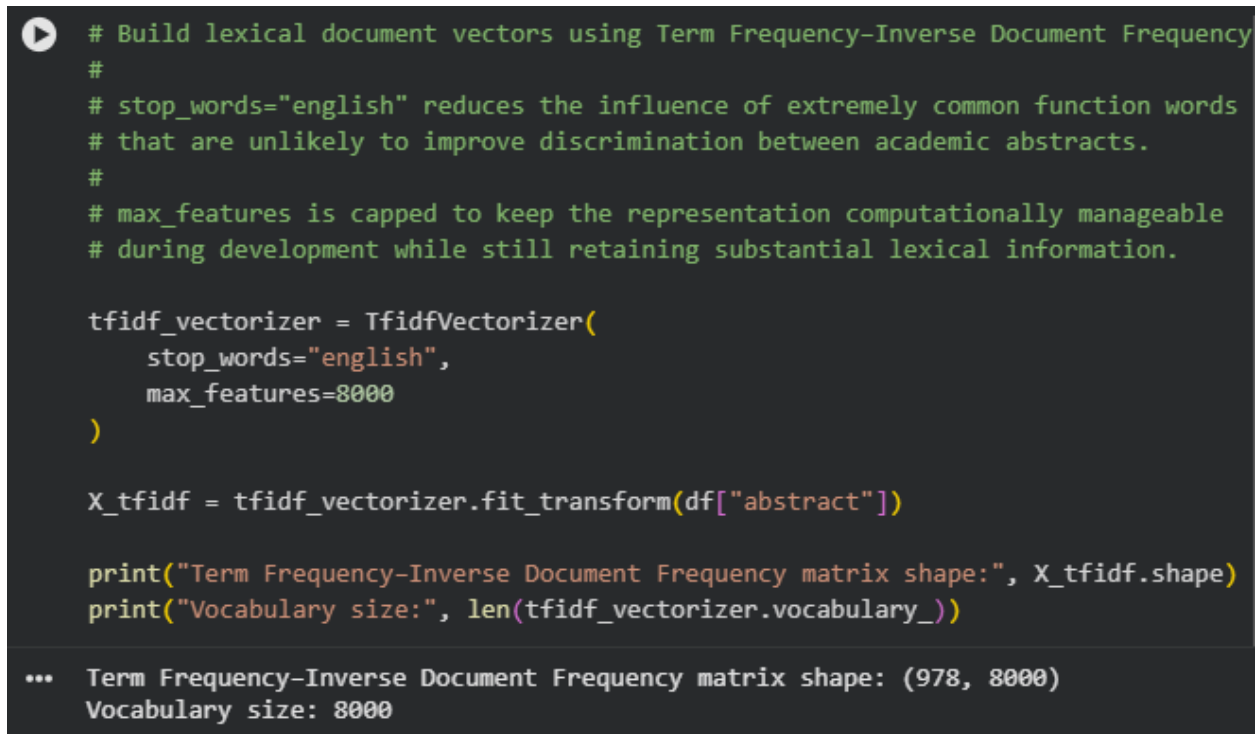
The choice of this implementation was justified by both theory and practicality. From a theoretical perspective, term frequency-inverse document frequency remains a defensible baseline because it models content-bearing vocabulary in a way that is transparent and well established in retrieval research. From a practical perspective, the method was lightweight and quick to compute, making it suitable for repeated experimentation and comparison against the more resource-intensive semantic branch (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

Once the sparse vectors were generated, cosine similarity was used to compute pairwise similarity scores between documents. Because the vectors were normalised, cosine similarity effectively captured directional closeness in the high-dimensional term space. For each seed document, similarity scores were calculated against the remaining corpus and then sorted in descending order to generate a ranked list of recommendations.

One strength of the lexical implementation was its simplicity and interpretability. Because the representation was based on visible weighted terms, the behaviour of the model was easier to reason about than the dense semantic model. This made it especially valuable as a baseline for comparison. However, the implementation also reflected the limitations discussed earlier: the

lexical branch depends heavily on surface word overlap and therefore cannot directly account for contextual semantic similarity when related papers use different vocabulary.

Figure 4.3 shows the implementation of the TF-IDF vectorisation stage used to construct the lexical document representation.



```
# Build lexical document vectors using Term Frequency-Inverse Document Frequency
#
# stop_words="english" reduces the influence of extremely common function words
# that are unlikely to improve discrimination between academic abstracts.
#
# max_features is capped to keep the representation computationally manageable
# during development while still retaining substantial lexical information.

tfidf_vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=8000
)

X_tfidf = tfidf_vectorizer.fit_transform(df["abstract"])

print("Term Frequency-Inverse Document Frequency matrix shape:", X_tfidf.shape)
print("Vocabulary size:", len(tfidf_vectorizer.vocabulary_))

... Term Frequency-Inverse Document Frequency matrix shape: (978, 8000)
Vocabulary size: 8000
```

Figure 4.3 TF-IDF vectorisation stage used to construct the lexical document representation

4.7 Implementation of the Semantic Recommendation Branch Using Sentence-BERT (SBERT) Embeddings

The semantic branch was implemented using a **transformer-based sentence embedding model** to convert document abstracts into dense vector representations. Unlike the lexical branch, which represents documents through weighted term occurrence, the semantic branch represents each document as a compact numerical embedding intended to capture contextual and conceptual meaning.

A pre-trained Sentence-BERT (SBERT) model, specifically the all-MiniLM-L6-v2 variant, was used to generate dense vector representations of the document abstracts. These vectors were then stored in a numerical array structure for efficient similarity computation. This stage was the most

computationally demanding part of the entire implementation, as embedding generation requires substantially more processing than term frequency-inverse document frequency vectorisation. This difference was expected and is consistent with the broader literature on transformer-based sentence embeddings, which shows that semantic models can offer more expressive representations at the cost of increased representation-time overhead (Reimers and Gurevych, 2019; Li *et al.*, 2020).

After embeddings had been generated, cosine similarity was applied in the same way as in the lexical branch. For a given seed document, the embedding vector was compared against the embeddings of the remaining documents, and the resulting similarity scores were sorted to produce a ranked recommendation list.

The implementation of this branch highlights one of the central tensions in the project. On the one hand, semantic embeddings are designed to identify relationships between documents even when the wording differs, making them especially promising for academic recommendation tasks. On the other hand, the generation of these embeddings requires more computational effort, which has implications for feasibility, scalability, and runtime. The implementation chapter does not yet evaluate which trade-off is preferable overall, but it does make clear that the semantic branch is qualitatively different from the lexical one in both representational richness and resource demand.

Figure 4.4 shows the implementation of the semantic branch using the Sentence-BERT model all-MiniLM-L6-v2 to generate dense document embeddings.

```

from sentence_transformers import SentenceTransformer

# Load a pretrained transformer-based sentence embedding model.
# This model is lightweight enough for Colab while still providing
# a meaningful semantic baseline for comparison.

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# Generate semantic embeddings for all abstracts.
# normalize_embeddings=True ensures that dot product is equivalent to cosine similarity,
# which keeps the comparison with the lexical branch methodologically consistent.

embeddings = embedding_model.encode(
    df["abstract"].tolist(),
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True
)

# Save embeddings so the computation does not need to be repeated unnecessarily.
np.save(os.path.join(OUTPUT DIR, "semantic_embeddings.npy"), embeddings)

```

Figure 4.4 SBERT embedding generation using the all-MiniLM-L6-v2 model for semantic document representation.

4.8 Similarity Computation and Recommendation Generation

After both sets of document representations had been created, the next implementation step was to generate recommendations. This stage was deliberately standardised across both pipelines in order to preserve the comparative integrity of the system.

For each selected seed paper, cosine similarity scores were computed between that paper and all other documents in the corpus. The resulting scores were then ranked from highest to lowest, and the top-k entries were treated as the recommendation output. This ranking logic was applied identically in both the lexical and semantic branches, ensuring that any later difference in recommendation quality would be attributable primarily to the representation method rather than to the ranking structure.

This standardisation was important. Recommendation experiments can be affected not only by how documents are represented, but also by how candidates are filtered, ranked, or truncated. By implementing a consistent top-k ranking mechanism across both branches, the project reduced the risk of introducing hidden asymmetries into the comparison.

The generation of ranked outputs also served a second purpose beyond recommendation itself: it produced the structured data needed for later testing and evaluation. In other words, recommendation generation in this chapter is not merely an end-user feature but also a necessary intermediate step that enables systematic metric calculation in Chapters 5 and 6.

Figure 4.5 presents the recommendation functions used to generate top-k ranked outputs for both the lexical and semantic branches.

```
def get_top_recommendations_tfidf(seed_index, top_n=5):
    similarity_scores = cosine_similarity(X_tfidf[seed_index], X_tfidf).flatten()

    # Prevent the seed document from recommending itself
    similarity_scores[seed_index] = -1

    ranked_indices = np.argsort(similarity_scores)[::-1][:top_n]

    output = df.loc[ranked_indices, ["title", "year", "citations"]].copy()
    output["doc_index"] = ranked_indices
    output["similarity_score"] = similarity_scores[ranked_indices]

    return output.reset_index(drop=True)

def get_top_recommendations_embeddings(seed_index, top_n=5):
    similarity_scores = embeddings @ embeddings[seed_index]
    similarity_scores[seed_index] = -1

    ranked_indices = np.argsort(similarity_scores)[::-1][:top_n]

    output = df.loc[ranked_indices, ["title", "year", "citations"]].copy()
    output["doc_index"] = ranked_indices
```

Figure 4.5 Recommendation ranking functions for the lexical and semantic branches of the system.

4.9 Exporting Outputs for Testing and Evaluation

The implementation workflow was designed to produce structured outputs for later testing and analysis. These included recommendation rankings, metric summaries, runtime measurements, dataset-level statistics, and visual outputs such as performance and runtime graphs. This was important because it linked implementation directly to the later analytical chapters and ensured that the system generated evidence as well as recommendations.

Storing outputs in organised formats such as comma-separated value files, numerical arrays, and image files also made later inspection and interpretation more straightforward. This helped ensure that results could be reviewed consistently during testing and evaluation.

4.10 Computational Considerations During Implementation

One practical observation during implementation was the difference in processing cost between the two pipelines. The lexical branch, based on term frequency-inverse document frequency, was comparatively fast to execute, whereas the semantic branch required noticeably more time during embedding generation, reflecting the greater computational demands of transformer-based sentence embedding models (Reimers and Gurevych, 2019; Li *et al.*, 2020).

At this stage, this difference is noted as an implementation consideration rather than a formal evaluative result. Its broader significance is discussed in Chapter 6, where runtime is examined alongside recommendation quality.

4.11 Implementation Challenges and Resolutions

Several practical challenges emerged during development. The first concerned data consistency, since the textual content used for representation had to remain formatted consistently and aligned with document indices throughout the workflow. This was important because any misalignment between documents, vectors, and rankings would have weakened the validity of the outputs.

A second challenge involved computational overhead in the semantic branch. Embedding generation required substantially more time and processing resources than lexical vectorisation. This was managed by using a moderate-sized filtered corpus and by generating and storing embeddings before repeated similarity comparisons.

A third challenge was maintaining comparability between the two pipelines. Because the lexical and semantic branches required different preprocessing strategies, there was a risk of introducing unfairness into the comparison. This was addressed by keeping the broader recommendation logic, ranking mechanism, and evaluation structure constant while allowing preprocessing to remain appropriate to each branch.

Finally, the implementation had to remain analytically useful rather than unnecessarily complex. Although additional metadata, citation signals, or hybrid ranking logic could have been

introduced, doing so would have made it harder to isolate the variable under investigation. The system therefore remained deliberately focused on clarity and controlled comparison.

5 Testing

5.1 Introduction

Chapter 4 described how the project design was translated into a functioning academic recommendation pipeline. Before the comparative performance of the lexical and semantic approaches can be interpreted, it is necessary to show that the implemented system operated correctly and consistently. In this project, testing refers to the structured process of verifying that the main pipeline components behaved as intended and that both recommendation branches could be executed reliably under the same conditions.

This distinction between testing and evaluation is important because testing establishes whether the system works, while evaluation examines how well it performs. Since the project compares two recommendation approaches, testing must first confirm that both branches function correctly before any comparative claim can be made.

Therefore, this Chapter focuses on a testing strategy based on key stages of the pipeline such as dataset loading, preprocessing, vector construction, similarity computation, ranking generation, and output storage. Because the project is a content-based recommendation experiment rather than a user-facing application, the most relevant tests were functional and pipeline-oriented rather than interface-based (Ammann and Offutt, 2016).

5.2 Purpose and Scope of Testing

The purpose of testing in this project was to establish that the implemented recommendation system functioned correctly at each major stage of execution. More specifically, testing aimed to confirm that the filtered OpenAlex-derived dataset could be loaded without structural error, that textual data could be transformed into both lexical and semantic representations, that similarity scores could be computed consistently, and that ranked recommendation outputs could be generated and exported in a stable form.

The scope of testing was deliberately focused on **system correctness and operational reliability** rather than comparative performance. In other words, testing addressed whether the system worked, not whether one recommendation method outperformed the other. This separation was important because the project needed to distinguish technical validity from

analytical judgement. The literature on software testing emphasises that testing is concerned with identifying faults and confirming intended behaviour, whereas broader performance interpretation belongs to later evaluative analysis (Ammann and Offutt, 2016).

Accordingly, the testing stage covered the following areas: data integrity, preprocessing consistency, vector generation, dimensional alignment, recommendation generation, output structure, and runtime capture. It did not attempt to determine final comparative superiority between the lexical and semantic methods, because that analytical task belongs to Chapter 6.

5.3 Testing Strategy

The project adopted a **layered functional testing strategy**. Rather than treating the recommendation pipeline as a single opaque system, testing was carried out across successive stages so that faults could be identified more clearly. This was appropriate because the implementation itself had been structured as a modular pipeline in Chapter 4. A modular system is easier to test because each component can be checked individually before attention shifts to end-to-end behaviour.

The strategy combined three complementary forms of verification. First, **component-level checks** were used to verify that individual stages behaved correctly, such as confirming that vectorisation produced the expected number of document representations. Second, **pipeline consistency checks** were used to ensure that outputs from one stage aligned correctly with the inputs required by the next stage. Third, **end-to-end functional checks** were used to verify that the full recommendation process could run from dataset input through to ranked output generation. These testing strategies are summarised and presented in Table 5.1 below.

This strategy was suitable for the project because it balanced practicality with technical rigour. The system did not require heavy interface testing or user acceptance testing, since it was not designed as a deployed application. Instead, the crucial issue was whether the research pipeline operated correctly and repeatedly. This kind of staged testing is consistent with established software testing practice, particularly for data-driven systems whose outputs depend on correct transformation across multiple processing stages (Ammann and Offutt, 2016).

Testing level	Purpose	Application in this project
Component-level testing	Check correctness of individual processing stages	Dataset loading, preprocessing, vector generation
Pipeline consistency testing	Check compatibility between stages	Document-index alignment, representation-to-ranking flow
End-to-end functional testing	Check complete system behaviour	Seed input to top-k recommendation output

Table 5.1: Testing strategy adopted in the project.

5.4 Testing Environment

Testing was carried out within the same notebook-based Python environment used for implementation. This ensured that the system was tested under the same practical conditions in which it had been developed. The main libraries involved in testing were Pandas, NumPy, scikit-learn, and sentence-transformers, since these libraries underpinned the dataset handling, representation generation, and similarity computation stages described previously. The notebook environment was useful because it allowed intermediate outputs to be inspected directly and made it easier to identify at which stage a problem emerged.

The testing environment also reflected the project’s practical scope. Since the recommendation system was implemented as a data-processing and ranking pipeline rather than a graphical application, testing focused on structured execution, numerical output, and workflow reliability rather than interface interaction. This is consistent with the project’s design and with the report template’s distinction between testing and evaluation.

5.5 Dataset and Input Validation Tests

The first testing stage focused on the input dataset. Since all later stages depended on the quality and consistency of the textual data, it was necessary to verify that the dataset had been loaded correctly and that the required fields were present and usable. This included checking that the number of records matched the expected filtered corpus size, that abstracts were available for all retained documents, and that the text field used for representation did not contain null values likely to break vectorisation or embedding generation.

These checks were important because data-related faults can produce misleading downstream behaviour. For example, if a document is retained without valid text content, then its representation may fail silently or produce an empty vector, affecting recommendation quality and potentially distorting the comparative results later on. Early testing therefore aimed to rule out basic structural problems before more complex recommendation behaviour was examined.

The dataset-level tests confirmed that the final working corpus contained **documents**, consistent with the filtered dataset profile established earlier, and that the document records used for representation were structurally aligned with the implementation pipeline.

5.6 Preprocessing and Representation Tests

The next stage of testing focused on the correctness of the representation pipelines.

For the lexical branch, the testing objective was to confirm that preprocessing and vectorisation produced a valid sparse representation for the full corpus. This involved checking that the term frequency-inverse document frequency matrix was generated without error, that the number of rows corresponded to the number of documents, and that the resulting representation could be passed successfully to cosine similarity computation. Since the lexical branch depends on structured text cleaning and weighted term mapping, these checks were necessary to confirm that the representation stage had not introduced mismatches between documents and vectors.

For the semantic branch, the main testing objective was to verify that every document produced an embedding and that the number of embeddings matched the number of records in the dataset. This was particularly important because semantic embedding generation is the most computationally expensive stage of the project and therefore the stage most vulnerable to interruptions or incomplete processing. The tests also checked that the embeddings could be stored and re-used for later similarity computation, rather than regenerated unnecessarily for each recommendation request.

A further issue tested at this stage was **index alignment**. Since both branches generate document representations that later feed into ranking, it was essential to confirm that the representation at a given position in the vector structure still corresponded to the correct document in the dataset.

Without this alignment, the recommendation rankings would be technically invalid even if the similarity computations themselves ran without error.

5.7 Similarity and Ranking Tests

Once representations had been confirmed, testing moved to the recommendation stage itself. The objective here was to verify that similarity scores could be generated for a chosen seed paper and that the ranking function returned the expected number of candidate recommendations in descending similarity order.

Several functional checks were important at this stage:

- The system needed to exclude or appropriately handle self-recommendation, since the seed paper would naturally be maximally similar to itself
- The top-k output was required to contain the specified number of recommendations.
- The ranking needed to remain stable and interpretable, ensuring that similarity scores and associated document identifiers were correctly aligned in the final output.
- The same ranking process needed to operate consistently across both the lexical and semantic branches without requiring separate downstream logic.

These checks were essential because recommendation generation is the main system function of the project. If the ranking mechanism were faulty, then later performance measures such as precision at k or normalised discounted cumulative gain at k would not be meaningful. The testing stage therefore ensured that the recommendation pipeline behaved correctly before any interpretation of relative recommendation quality took place.

Table 5.2 presents a summary of the functional test cases used to verify the correctness of the recommendation pipeline across its main processing stages.

Test case	Expected result	Outcome
Load filtered dataset	Dataset loads successfully with expected record count	Passed
Validate text field	No null abstracts in active corpus	Passed
Generate lexical vectors	Sparse matrix created for all documents	Passed
Generate semantic embeddings	Embedding produced for each document	Passed
Compute similarity scores	Scores returned for seed against candidate corpus	Passed
Generate top-k recommendations	Ranked list returned with correct length	Passed
Export output files	Results saved in structured format	Passed

Table 5.2: Sample functional test cases used in the project.

An example of the generated recommendation output is shown in Figure 5.1 illustrating the differences between lexical and semantic similarity-based recommendations for a selected seed paper.

```

SEED PAPER:
IMAGE ARTIFACTS IN OPTICAL COHERENCE TOMOGRAPHY ANGIOGRAPHY
=====

LEXICAL (TF-IDF) RECOMMENDATIONS:
                                     title  similarity_score
0  Compression Artifacts Reduction by a Deep Conv...    0.200490
1  Infrared and Visible Image Fusion via Decoupli...    0.156058
2  Clearing the Skies: A Deep Network Architectur...    0.135399
3  EEG artifact removal-state-of-the-art and guid...    0.129430
4  Deep Supervised Hashing for Fast Image Retrieval    0.117120

SEMANTIC (EMBEDDING) RECOMMENDATIONS:
                                     title  similarity_score
0                X-ray computed tomography    0.390391
1  The effectiveness of virtual and augmented rea...    0.378229
2  Standardized image interpretation and post-pro...    0.332326
3  Measuring the optimal exposure for single part...    0.318808
4  A review of medical image data augmentation te...    0.307639

```

Figure 5.1 Example top-k recommendation output comparing lexical (TF-IDF) and semantic (SBERT) approaches for a selected seed paper.

5.8 Output and Storage Verification

The project also required testing of the exported outputs used later for performance analysis. This included recommendation rankings, runtime summaries, evaluation files, and visual outputs such as plots. The purpose of this stage was not to judge the meaning of those outputs, but simply to confirm that they were created successfully, stored in the expected format, and could be used in subsequent chapters without loss of structural consistency.

This stage was particularly important because the project depends on a chain of evidence. The evaluation chapter is only credible if the preceding implementation and testing stages show that the stored outputs genuinely reflect successful system execution. Verifying output creation therefore formed part of the overall testing strategy rather than being treated as an afterthought.

5.9 Runtime Analysis

Although full comparison of computational cost belongs in Chapter 6, basic runtime checks were also relevant during testing. These checks were not used to determine comparative superiority, but to confirm that the two branches executed in a plausible and stable manner. In practice, the testing stage confirmed that lexical vector construction and recommendation generation completed quickly, while semantic vector construction was substantially slower. A detailed runtime comparison supporting this observation is presented in Figure 6.1 in Chapter 6. This observation was consistent with the implementation expectations for transformer-based sentence embedding models, whose greater representational richness typically comes with higher computational overhead (Reimers and Gurevych, 2019).

The purpose of these runtime checks was therefore limited but useful. They established that the system’s computational behaviour was internally coherent and that the runtime figures later reported in the evaluation chapter were not the result of failed or abnormal execution.

5.10 Testing Outcomes and Interpretation

Overall, the testing stage showed that the recommendation system functioned correctly across its main stages. The dataset could be loaded and validated successfully, the two representation

pipelines could be executed without structural failure, similarity scores could be computed, ranked recommendations could be generated, and outputs could be exported for later analysis. These outcomes indicate that the implemented system was technically stable enough to support the formal comparative evaluation presented in the next chapter.

It is important to emphasise that these results should not yet be interpreted as evidence that one similarity method is better than the other. The purpose of testing was narrower. It established that both methods could be executed correctly within the same project framework and that the resulting outputs were suitable for further analysis. This separation between operational verification and comparative judgement is central to the logic of the report and is consistent with the final report structure provided for the project.

5.11 Limitations of the Testing Stage

Although the testing strategy was appropriate for the scope of the project, it also had clear limits. First, testing was conducted within an experimental notebook-based environment rather than a deployed production system. This means that the testing process focused on pipeline correctness and research reliability rather than on deployment-level robustness, concurrency, or user-interface behaviour. Second, relevance itself was not tested at this stage, since that belongs to evaluation. Third, the project did not include formal automated unit-testing suites in the style of large-scale software engineering projects. Instead, it used structured functional and pipeline verification appropriate to a data-driven academic experiment.

These limitations do not invalidate the testing stage. Rather, they reflect the nature of the project. The implemented system is a research-oriented recommendation pipeline, and the testing strategy was designed accordingly. What matters is that the tests were sufficient to confirm technical correctness before the later analytical claims were made.

Having established that the system functions correctly, the next chapter evaluates the comparative performance of the two approaches.

6 Evaluation and Discussion

6.1 Introduction

Chapter 5 showed that the recommendation pipeline functioned correctly across its main stages. This chapter focuses instead on how well the system performed in relation to the project objectives. It evaluates the comparative effectiveness of the lexical and semantic approaches within the academic recommendation task and interprets the results in relation to computational feasibility and prior research.

Unlike testing, which verified the correctness of the pipeline, evaluation examines the quality and implications of the resulting recommendation behaviour. This chapter therefore analyses the outputs through ranking-based metrics and runtime comparison before relating the findings to the scholarly recommendation literature. Offline evaluation is especially appropriate here because it remains the most common and practical form of evaluation in scholarly recommendation research, even though it cannot fully capture user-centred judgement or live-system behaviour (Kreutz and Schenkel, 2022; Khadka and Sthapit, 2025).

The central question is whether the semantic method improved recommendation quality relative to the lexical baseline, and whether any improvement came with a meaningful computational cost. These questions reflect the broader trade-off identified earlier between the efficiency of lexical methods and the greater expressive power of semantic approaches (Zhang *et al.*, 2023; Khadka and Sthapit, 2025)

6.2 Evaluation Purpose and Criteria

The evaluation was designed to assess how effectively each method generated relevant academic recommendations under the same pipeline conditions. To preserve the comparative logic of the project, both recommendation branches were evaluated using the same dataset, the same recommendation task, and the same broad ranking framework. The criteria used for evaluation were selected to reflect the practical demands of academic recommendation, where users typically inspect only the highest-ranked results rather than the full candidate set.

Three ranking-oriented measures were particularly important: **precision at k**, **recall at k**, and **normalised discounted cumulative gain at k**. Recent reviews of scholarly publication recommender systems identify these as among the most common and useful offline measures for comparing recommendation quality, especially in top-N recommendation settings (Khadka and Sthapit, 2025). Likewise, broader surveys of scholarly recommenders note that ranking quality remains central because recommendation systems are valued not simply for finding relevant items, but for ranking them well enough that useful items appear near the top (Zhang *et al.*, 2023).

In addition to ranking quality, the evaluation also considers **runtime**. This is a necessary part of the project because one of the stated objectives was to examine the practical trade-offs between the two approaches. A method that yields better ranking quality but requires much greater computational effort may still be valuable, but its suitability depends on the intended system environment. Including runtime therefore prevents the evaluation from collapsing into a narrow accuracy-only comparison.

6.3 Evaluation Method

The evaluation followed an **offline comparative design**, which is appropriate for early-stage algorithm comparison and consistent with prevailing practice in scholarly recommendation research (Kreutz and Schenkel, 2022; Khadka and Sthapit, 2025). Both the lexical and semantic methods were run on the same OpenAlex-derived corpus of 978 Computer Science documents. The dataset characteristics, including average abstract length and corpus size, were described earlier and provided a sufficiently rich but computationally manageable basis for recommendation analysis.

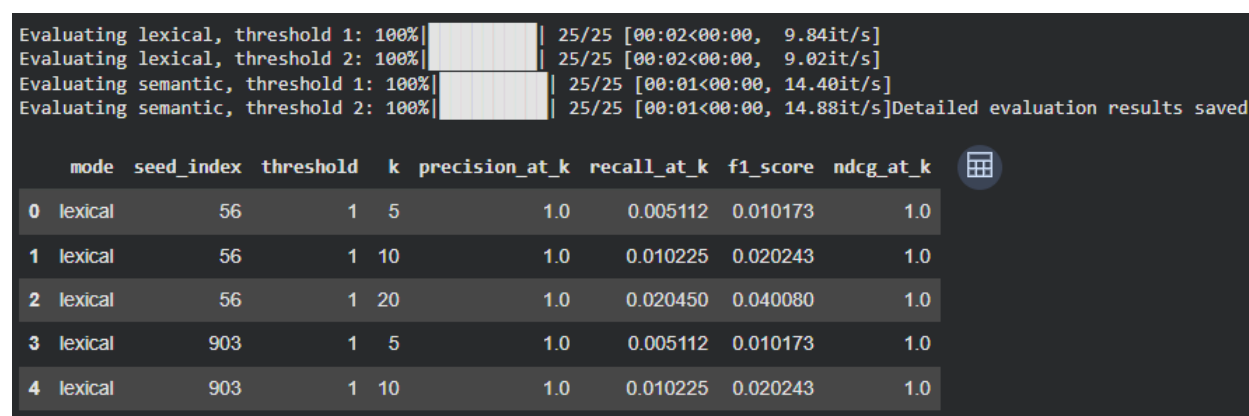
For each branch, document representations were constructed, cosine similarity was used to compute similarity scores, and top-k ranked recommendation lists were produced for the selected evaluation settings. The resulting rankings were then assessed using precision at k and normalised discounted cumulative gain at k across different cut-off values as seen in Figure 6.1. The visual outputs shared from the implementation indicate results reported at **k = 5, 10, and 20**, and under at least two relevance-threshold settings. Two relevance-threshold settings were used

to examine how stricter and more relaxed definitions of relevance affected recommendation performance. In this project, relevance is based on the number of shared OpenAlex concept identifiers between documents. A threshold of 1 represents a more permissive condition, while a threshold of 2 requires greater conceptual overlap.

These two thresholds were selected to balance evaluation depth and stability. While higher thresholds (e.g., 3 or more) could be applied, they would significantly reduce the number of relevant documents in the dataset, making the evaluation less reliable. The chosen thresholds therefore provide a practical comparison between lenient and stricter relevance conditions.

The use of multiple cut-off values is important. A recommendation method may perform strongly at very short lists but decline as the recommendation list expands. Evaluating performance across different k values therefore provides a clearer picture of ranking stability rather than a single-point snapshot. This is particularly relevant in academic recommendation because some users may inspect only a few results, while others may explore larger related-work sets. All evaluation metrics were averaged across multiple seed documents to ensure stability and reduce variance in the results.

Figure 6.1 illustrates a detailed evaluation analysis output from the comparison between lexical and semantic embeddings



The image shows a terminal window with evaluation progress bars and a table of results. The progress bars indicate that all evaluations (lexical and semantic at thresholds 1 and 2) are 100% complete. The table below summarizes the results for lexical embeddings.

	mode	seed_index	threshold	k	precision_at_k	recall_at_k	f1_score	ndcg_at_k
0	lexical	56	1	5	1.0	0.005112	0.010173	1.0
1	lexical	56	1	10	1.0	0.010225	0.020243	1.0
2	lexical	56	1	20	1.0	0.020450	0.040080	1.0
3	lexical	903	1	5	1.0	0.005112	0.010173	1.0
4	lexical	903	1	10	1.0	0.010225	0.020243	1.0

Figure 6.1 Detailed evaluation results

6.4 Quantitative Results

6.4.1 Runtime comparison

The runtime chart shows a very clear computational imbalance between the two approaches. The most visually dominant result is that **semantic vector construction required far more time** than any other stage of the pipeline. In contrast, lexical vector construction and recommendation generation appear comparatively lightweight. Based on the plotted values, semantic vector construction took roughly **116 seconds**, while the remaining stages were visually close to zero by comparison. This indicates that the main computational burden of the semantic pipeline lay not in ranking itself, but in the production of the dense document representations.

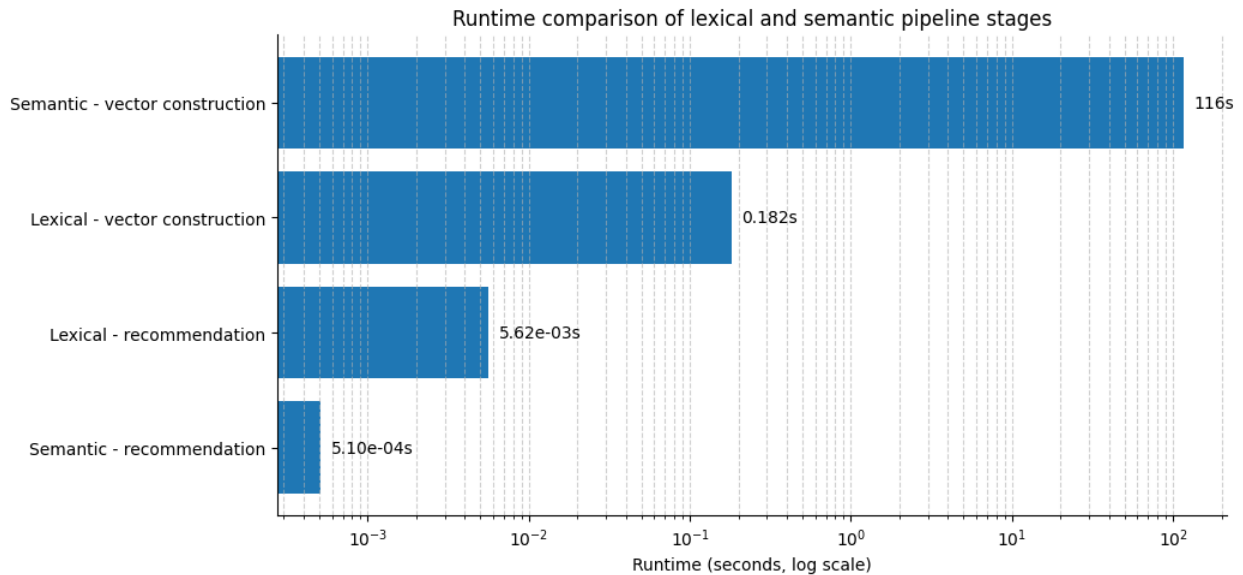


Figure 6.2: Runtime comparison of lexical and semantic vector construction and recommendation generation stages (Project output).

This finding confirms, in practical terms, the trade-off discussed in the literature. Sentence-transformer style embedding pipelines are attractive for semantic search because they support efficient similarity comparison once embeddings have been created, but the up-front embedding stage is substantially more resource-intensive than classical lexical vectorisation (Reimers and Gurevych, 2019; Sentence Transformers, 2026). In the context of this project, the runtime evidence therefore supports the view that semantic methods may improve recommendation quality, but they do not do so without computational cost.

6.4.2 Precision at k

As illustrated in Figure 6.3, the precision plot indicates that the semantic method consistently outperformed the lexical method across the reported cut-off values. Under the threshold-1 setting, semantic precision remained at 1.00 across $k = 5, 10$, and 20 , which reflects the permissive relevance definition under threshold = 1. Under the threshold-2 setting, the semantic method remained higher, decreasing from approximately 0.97 to approximately 0.91, while the lexical method declined more sharply from approximately 0.88 at $k = 5$ to approximately 0.82 at $k = 20$.

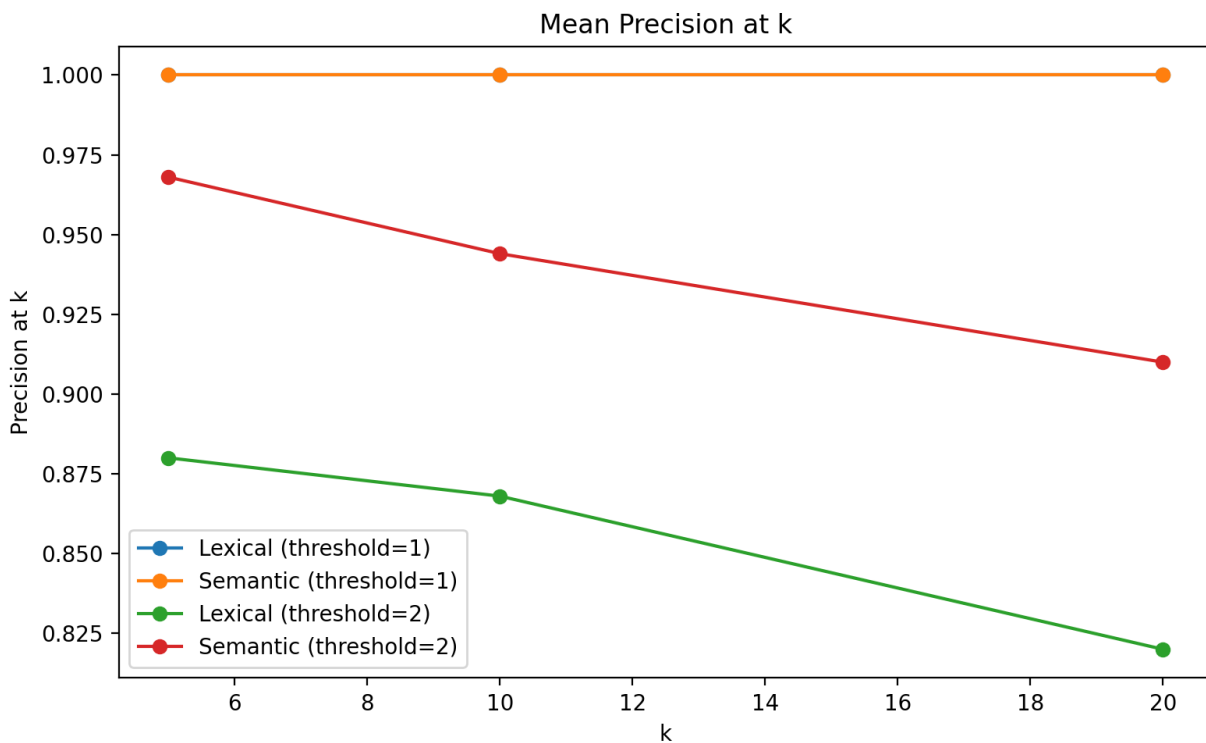


Figure 6.3: Mean precision at k for lexical and semantic recommendation approaches under different relevance thresholds (Project output).

Two key observations emerge from this pattern. First, the semantic method is not only stronger overall but also more stable as k increases. Second, the lexical method shows a clearer decline as the recommendation list expands, indicating reduced effectiveness in maintaining relevance at deeper ranking levels. This suggests that while lexical matching may remain useful for near-neighbour retrieval, semantic representation is better suited to sustaining thematic relevance across a broader range of recommended papers.

6.4.3 Normalised discounted cumulative gain at k

As illustrated in Figure 6.4, the pattern in normalised discounted cumulative gain at k closely reinforces the precision results. Under the threshold-1 setting, semantic normalised discounted cumulative gain remained at 1.00 across all reported cut-off values due to the permissive relevance definition, making threshold = 2 more informative. Under the threshold-2 setting, semantic performance remained substantially above the lexical baseline, declining from approximately 0.96 at $k = 5$ to approximately 0.92 at $k = 20$. In contrast, the lexical method under threshold-2 declined more sharply, from approximately 0.89 to approximately 0.84.

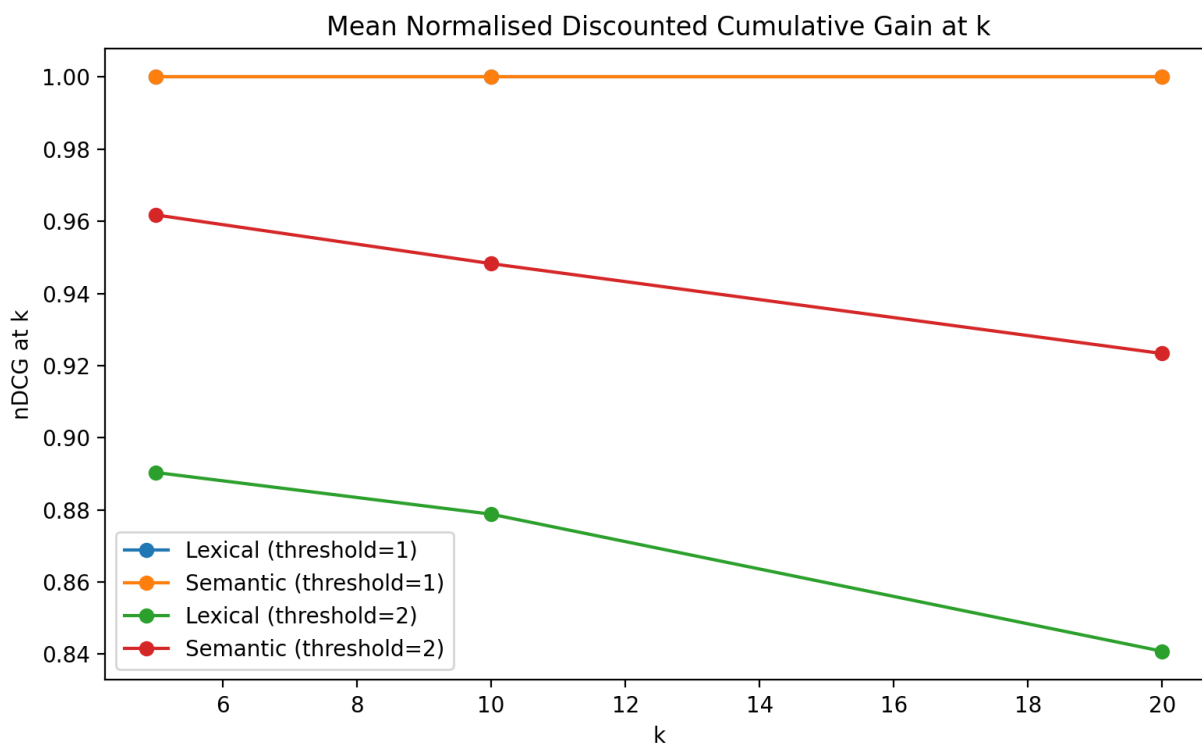


Figure 6.4: Mean normalised discounted cumulative gain at k for lexical and semantic recommendation approaches under different relevance thresholds (Project output).

This is an important result because normalised discounted cumulative gain is sensitive not only to whether relevant documents appear in the recommendation list, but also to **where** they appear. The semantic approach therefore did not merely retrieve relevant items; it appears to have ranked them more effectively near the top of the list. Since top-ranked results are especially important in academic discovery, this strengthens the case for the semantic method beyond raw retrieval alone.

Table 6.1 summarises the main patterns observed across the evaluation results and provides a compact comparison between the lexical and semantic approaches before the findings are discussed in relation to prior studies.

Aspect	Lexical approach	Semantic approach	Overall interpretation
Precision at k	Lower and declines more clearly with k	Higher and more stable across k values	Semantic stronger
Normalised discounted cumulative gain at k	Lower positional ranking quality	Higher positional ranking quality	Semantic ranks relevant items earlier
Runtime	Lightweight and faster to execute	Much more expensive during vector construction	Lexical more efficient

Table 6.1: Summary of observed performance patterns across lexical and semantic recommendation approaches.

6.4.4 Dataset context

The dataset profile helps contextualise these results. The working corpus contained **978 documents**, with an **average abstract length of 168.23 words** and a **median abstract length of 162 words**. This means that the recommendation system operated over documents that contained enough textual substance for both lexical and semantic analysis, while still remaining computationally feasible within the project environment. The abstract-length distribution you shared also suggests a generally moderate-length academic corpus with a right-skewed tail of longer abstracts, which is a plausible environment for testing whether semantic representations can preserve conceptual similarity better than sparse word-based vectors.

6.5 Discussion of Findings

Taken together, the results indicate that the semantic recommendation approach was more effective than the lexical baseline in this project, but also more computationally expensive. It achieved stronger precision and stronger normalised discounted cumulative gain, suggesting that

transformer-based sentence embeddings captured conceptual similarity more effectively than term frequency-inverse document frequency. This is consistent with the theoretical distinction developed earlier: lexical methods rely on shared vocabulary, whereas semantic embeddings are designed to preserve contextual similarity even when wording differs.

A likely explanation is that the academic abstracts in the dataset contained enough conceptual variation for semantic representation to provide a real advantage. Where related Computer Science papers described similar methods or problems using different terminology, a surface-oriented lexical model would naturally understate their similarity. At the same time, the runtime results prevent a simplistic conclusion. The semantic method improved ranking quality, but at a much higher resource cost. A more accurate conclusion is therefore that it offered better recommendation quality at greater computational expense. This matters because in low-resource settings, or in systems requiring rapid indexing, the lexical baseline may still remain attractive.

A second important finding concerns ranking stability. The lexical method declined more clearly as k increased, whereas the semantic method remained relatively stronger across longer recommendation lists. This suggests that semantic representation preserved recommendation quality more effectively beyond the very top results, which is valuable in academic discovery where users often move from an initial shortlist to a broader reading set.

6.6 Comparison with Existing Studies

The findings of this project align with the existing literature in several ways. First, they support the broader view that content-based recommendation remains central in scholarly recommendation systems, particularly where rich behavioural data are unavailable and recommendation depends mainly on document similarity (Zhang *et al.*, 2023; Khadka and Sthapit, 2025).

Second, the results support the view that semantic representation can improve recommendation quality relative to sparse lexical methods, especially where conceptual similarity is not fully captured by shared terms. Recent work on sentence embeddings and transformer-based recommendation shows that such models are well suited to semantic search and thematic matching once efficient vector comparison is established (Reimers and Gurevych, 2019;

Colangelo *et al.*, 2025). The stronger precision and normalised discounted cumulative gain observed here are therefore consistent with prior research.

Third, the runtime findings also align with existing work. Sentence-embedding methods make large-scale semantic comparison practical, but embedding generation remains a distinct and computationally demanding stage (Reimers and Gurevych, 2019). The present project confirms that even in a moderate-sized corpus, representation can dominate runtime.

At the same time, this project differs from many published systems in methodological scope. Much of the scholarly recommendation literature evaluates more complex architectures incorporating citation networks, multi-stage pipelines, or hybrid methods. For example, Kanwal and Amjad (2024) report a citation-network-based research paper recommendation system using multiple features over a much larger citation graph. Such systems may achieve strong results, but they also make it harder to isolate the contribution of a particular representation method. The present project is narrower, but that narrowness is also its strength: it provides a clearer comparison between lexical and semantic text representation alone.

Finally, the project reflects a broader limitation repeatedly noted in review literature: the field still relies heavily on offline evaluation because user studies and online evaluation are costly and difficult to conduct. In that respect, the project is consistent with prevailing practice, while also acknowledging the limits of what offline ranking metrics can capture (Khadka and Sthapit, 2025).

6.7 Limitations of the Evaluation

Although the evaluation provides clear comparative evidence, it also has important limitations. First, the project used an offline evaluation design rather than a live user study or deployment-based assessment. This is common in scholarly recommendation research, but it means the results reflect ranking quality rather than direct user satisfaction (Khadka and Sthapit, 2025).

Second, the evaluation was carried out on a moderate-sized, domain-focused dataset. The corpus was appropriate for controlled experimentation, but the findings should not be generalised uncritically to all disciplines or large-scale production systems.

Third, the project compared only two similarity paradigms within a content-based setting. It did not incorporate citation-context features, graph signals, collaborative filtering, or hybrid

reranking. This limits its scope, but it was also a deliberate choice in order to isolate the role of similarity measurement.

Fourth, some results were interpreted primarily from project plots rather than from fully tabulated benchmark values in the chapter text. This does not invalidate the findings, but it suggests that a compact summary table of the exact metric values would strengthen the final presentation. Precision_at_k and ndcg_at_k were prioritised for clearer comparison of top-ranked recommendation quality

6.8 Implications for the Project Objectives

The evaluation results can now be considered against the original project objectives.

The objective of designing a controlled recommendation pipeline was achieved, since both methods were implemented and evaluated under the same broader framework. The objective of implementing a lexical similarity baseline was also achieved, and the resulting branch provided a meaningful point of comparison rather than a trivial placeholder. The semantic branch based on transformer-derived sentence embeddings was likewise implemented successfully and produced measurable outputs suitable for direct comparison.

Most importantly, the objective of comparing recommendation effectiveness was achieved. The semantic method consistently outperformed the lexical baseline in the reported ranking measures. The objective of examining computational trade-offs was also satisfied, since the runtime results demonstrated a very clear representation-time cost associated with the semantic approach.

Taken together, this means the project does not simply conclude that semantic methods are stronger. It concludes something more useful: **semantic methods appear to improve recommendation quality in this academic recommendation setting, but they do so at a substantial computational cost, especially during embedding generation.** That is a more balanced and practically meaningful outcome than a one-dimensional accuracy claim.

This chapter showed that the semantic approach outperformed the lexical baseline in recommendation quality, but at a much higher computational cost. These findings are now brought together in the concluding chapter.

7 Conclusion and Future Work

7.1 Project Summary and Main Findings

This project set out to compare lexical and semantic similarity measurement techniques within a content-based academic recommendation system. The study was motivated by the growing difficulty of academic literature discovery and by the need to understand whether newer semantic methods offer meaningful advantages over traditional lexical approaches in scholarly recommendation contexts (Zhang *et al.*, 2023; Kreutz and Schenkel, 2022).

To address this aim, the project developed a controlled recommendation pipeline using an OpenAlex-derived dataset of 978 Computer Science documents. Two recommendation approaches were implemented within the same overall framework: a lexical baseline based on term frequency-inverse document frequency and a semantic approach based on transformer-based sentence embeddings. By keeping the dataset, recommendation task, similarity function, and evaluation framework stable, the project was able to compare the two approaches under consistent conditions.

The results showed that the semantic approach produced stronger recommendation quality than the lexical baseline across the reported ranking measures. In particular, the semantic method achieved higher precision at k and higher normalised discounted cumulative gain at k , indicating that it was more effective at placing relevant papers near the top of the recommendation list. However, the runtime findings also showed that this improved recommendation quality came with a substantially higher computational cost, especially during embedding generation.

Taken together, these findings suggest that semantic similarity offers a meaningful advantage for academic recommendation in this project setting, but that this advantage must be interpreted alongside practical considerations of efficiency and resource demand. The project therefore does not conclude that one method is universally superior in every respect. Instead, it shows that semantic similarity improved ranking effectiveness, while lexical similarity remained simpler and more computationally efficient.

7.2 Key Contributions and Limitations

The main contribution of the project lies in providing a controlled comparison between two important similarity paradigms within the same academic recommendation pipeline. This is valuable because much of the scholarly recommendation literature evaluates broader systems in which multiple components change simultaneously, making it harder to isolate the effect of representation choice itself (Kreutz and Schenkel, 2022; Zhang *et al.*, 2023).

A second contribution is the use of an OpenAlex-derived dataset to support a realistic and transparent recommendation experiment. The use of open scholarly metadata strengthens the practical relevance of the work and supports a clearer connection between implementation and reproducible academic research (Priem, Piwowar and Orr, 2022).

The project also has clear limitations. The evaluation was conducted on a moderate-sized, domain-focused dataset and used an offline ranking framework rather than direct user-centred assessment. In addition, the system remained deliberately content-based and did not incorporate collaborative, citation-network, or hybrid recommendation signals. These limitations do not weaken the value of the findings, but they define the scope within which the conclusions should be interpreted.

7.3 Future Work and Final Remarks

A direction for future work would be to test the system on a larger and more diverse scholarly dataset in order to examine whether the comparative advantage of semantic similarity remains consistent across other disciplines. A second useful extension would be to investigate whether the computational cost of the semantic branch can be reduced through more efficient embedding or retrieval strategies.

Overall, the project shows that similarity measurement is a central design decision in academic recommendation systems. Within the scope of this study, semantic similarity improved recommendation quality, while lexical similarity remained more efficient and easier to compute. The project therefore provides a balanced conclusion that is both technically relevant and practically meaningful for the design of academic recommendation tools.

8 References

- Ammann, P. and Offutt, J. (2016) *Introduction to Software Testing*. 2nd edn. Cambridge: Cambridge University Press. Available at: <https://www.cambridge.org/highereducation/books/introduction-to-software-testing/95E57CCADEA697EC8594F03729F47311>
- Colangelo, M.T., Meleti, M., Guizzardi, S., Calciolari, E. and Galli, C. (2025) ‘A Comparative Analysis of Sentence Transformer Models for Automated Journal Recommendation Using PubMed Metadata’, *Big Data and Cognitive Computing*, 9(3), p. 67. Available at: <https://www.mdpi.com/2504-2289/9/3/67>
- Khadka, A. and Sthapit, S. (2025) ‘A Review on Scholarly Publication Recommender Systems: Features, Approaches, Evaluation, and Open Research Directions’, *Informatics*, 12(4), p. 108. Available at: <https://www.mdpi.com/2227-9709/12/4/108>
- Kanwal, T. and Amjad, T. (2024) ‘Research paper recommendation system based on multiple features from citation network’, *Scientometrics*. Available at: <https://link.springer.com/article/10.1007/s11192-024-05109-w>
- Kreutz, C.K. and Schenkel, R. (2022) ‘Scientific paper recommendation systems: a literature review of recent publications’, *International Journal on Digital Libraries*, 23(4), pp. 335–369. Available at: <https://doi.org/10.1007/s00799-022-00339-w>
- Li, B., Zhou, H., He, J., Wang, M., Yang, Y. and Li, L. (2020) ‘On the Sentence Embeddings from Pre-trained Language Models’, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Stroudsburg, PA: Association for

Computational Linguistics, pp. 9119–9130. Available at:

<https://doi.org/10.18653/v1/2020.emnlp-main.733>

Priem, J., Piwowar, H. and Orr, R. (2022) ‘OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts’, *arXiv*. Available at:

<https://doi.org/10.48550/arXiv.2205.01833>

Putra, A.P., Putri, D.P.S. and Wiranatha, A.A.K.A.C. (2025) ‘Scientific Paper Recommendation System: Application of Sentence Transformers and Cosine Similarity Using arXiv Data’, *Journal of Applied Informatics and Computing*. Available at:

<https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/9766>

Reimers, N. and Gurevych, I. (2019) ‘Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks’, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Stroudsburg, PA: Association for Computational Linguistics, pp. 3982–3992. Available at: <https://aclanthology.org/D19-1410/>

scikit-learn (2026a) *TfidfVectorizer*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (Accessed: 30 March 2026).

scikit-learn (2026b) *cosine_similarity*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html (Accessed: 30 March 2026).

Sentence Transformers (2026) *Semantic Search*. Available at: https://sbert.net/examples/sentence_transformer/applications/semantic-search/README.html (Accessed: 30 March 2026).

Zhang, Z., Patra, B.G., Yaseen, A., Zhu, J., Sabharwal, R., Roberts, K., Cao, T. and Wu, H. (2023) ‘Scholarly recommendation systems: a literature survey’, *Knowledge and Information Systems*, 65, pp. 4433–4478. Available at: <https://doi.org/10.1007/s10115-023-01901-x>

9 Appendices

Appendix A: Key Implementation Codes

```
def get_top_recommendations_tfidf(seed_index, top_n=5):  
    similarity_scores = cosine_similarity(X_tfidf[seed_index], X_tfidf).flatten()  
  
    # Prevent the seed document from recommending itself  
    similarity_scores[seed_index] = -1  
  
    ranked_indices = np.argsort(similarity_scores)[::-1][:top_n]  
  
    output = df.loc[ranked_indices, ["title", "year", "citations"]].copy()  
    output["doc_index"] = ranked_indices  
    output["similarity_score"] = similarity_scores[ranked_indices]  
  
    return output.reset_index(drop=True)  
  
def get_top_recommendations_embeddings(seed_index, top_n=5):  
    similarity_scores = embeddings @ embeddings[seed_index]  
    similarity_scores[seed_index] = -1  
  
    ranked_indices = np.argsort(similarity_scores)[::-1][:top_n]  
  
    output = df.loc[ranked_indices, ["title", "year", "citations"]].copy()  
    output["doc_index"] = ranked_indices
```


Sentence-BERT (SBERT) model

```
# Load a pretrained transformer-based sentence embedding model.
# This model is lightweight enough for Colab while still providing
# a meaningful semantic baseline for comparison.

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# Generate semantic embeddings for all abstracts.
# normalize_embeddings=True ensures that dot product is equivalent to cosine similarity,
# which keeps the comparison with the lexical branch methodologically consistent.

embeddings = embedding_model.encode(
    df["abstract"].tolist(),
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True
)

# Save embeddings so the computation does not need to be repeated unnecessarily.
np.save(os.path.join(OUTPUT_DIR, "semantic_embeddings.npy"), embeddings)

print("Semantic embedding matrix shape:", embeddings.shape)
```

Appendix B: Additional Outputs

Lexical vs Semantic Embeddings Recommendation Output Example

```
SEED PAPER:
Path Planning and Tracking for Vehicle Collision Avoidance Based on Model Predictive Control With Multiconstraints
=====

LEXICAL (TF-IDF) RECOMMENDATIONS:
      title  similarity_score
0  Convolutional Features for Correlation Filter ...    0.160274
1  Planning and Decision-Making for Autonomous Ve...    0.153633
2      Direct and Indirect Effects    0.123117
3  HOTA: A Higher Order Metric for Evaluating Mul...    0.118170
4  Detection and Tracking Meet Drones Challenge    0.116262

SEMANTIC (EMBEDDING) RECOMMENDATIONS:
      title  similarity_score
0  Planning and Decision-Making for Autonomous Ve...    0.513683
1  UAV-Enabled Intelligent Transportation Systems...    0.354237
2  Review on model predictive control: an enginee...    0.327357
3  FAST-LIO: A Fast, Robust LiDAR-Inertial Odomet...    0.319415
4  Artificial intelligence, machine learning and ...    0.290422

--- ANALYSIS NOTE ---
Lexical recommendations rely more heavily on shared terminology, while semantic recommendations are better suited to concept-level relatedness.
```

Runtime Comparison Output

Batches: 100%

31/31 [01:56<00:00, 2.14s/it]

	Method	Time in seconds	
0	Lexical vector construction	0.182168	
1	Semantic vector construction	116.219206	
2	Lexical recommendation generation	0.005623	
3	Semantic recommendation generation	0.000510	

Meeting Logs

Meeting Logs

Name: Alvin Muinde

Student Number: M00899452

Date	Comments	Signature
20/01/26	Discussing and choosing topic	
03/02/26	Discussion on literature review and first steps	
18/02/26	Discussion on the evaluation and analysis plan	
03/03/26	Discussion on the Implementation Chapter	
17/03/26	Got feedback on coursework 1 and discussed the Implementation Chapter of the report	
31/03/26	Discussion on the Evaluation Chapter of the report and the project progression	

Ethics Form

Research Ethics Screening Form for Students

(For students on taught programmes only, NOT for PGRs - MRes/MPhil/PhD)

Middlesex University is concerned with protecting the rights, health, safety, dignity, and privacy of its research participants. It is also concerned with protecting the health, safety, rights, and academic freedom of its students and with safeguarding its own reputation for conducting high quality, ethical research.

This Research Ethics Screening Form will enable taught students to self-assess and determine whether their research requires ethical review and approval via the online Ethics Monitor before commencing their study. Supervisors must approve this form after consultation with students.

Student Name:	Alvin Fadhill Mambo Muinde	Email: am3726@live.mdx.ac.uk
Research project title:	A Comparative Analysis of Similarity Measurement Techniques for an Academic Recommendation System	
Programme of study/module:	CST3990 Undergraduate Individual Project	
Supervisor Name:	Dr Gina Soopramanien	Email: g.soopramanien@mdx.ac.mu

Please answer whether your research/study involves any of the following given below:		
1. "ANIMALS or animal parts.	☐ Yes	☒ No
2. "CELL LINES (established and commercially available cells - biological research).	☐ Yes	☒ No
3. "CELL CULTURE (Primary: from animal/human cells- biological research).	☐ Yes	☒ No
4. "CLINICAL Audits or Assessments (e.g. in medical settings).	☐ Yes	☒ No
5. "CONFLICT of INTEREST or lack of IMPARTIALITY. If unsure see "Code of Practice for Research" (Sec 3.5) at: https://unihub.mdx.ac.uk/study/spotlights/types/research-at-middlesex/research-ethics	☐ Yes	☒ No
6. "DATA to be used that is not freely available (e.g. secondary data needing permission for access or use).	☐ Yes	☒ No
7. "DAMAGE (e.g., to precious artefacts or to the environment) or present a significant risk to society).	☐ Yes	☒ No
8. "EXTERNAL ORGANISATION – research carried out within an external organisation or your research is commissioned by a government (or government body).	☐ Yes	☒ No
9. "FIELDWORK (e.g., biological research, ethnography studies).	☐ Yes	☒ No
10. "GENETICALLY MODIFIED ORGANISMS (GMOs) (biological research).	☐ Yes	☒ No
11. "GENE THERAPY including DNA sequenced data (biological research).	☐ Yes	☒ No
12. "HUMAN PARTICIPANTS – ANONYMOUS Questionnaires (participants not identified or identifiable).	☐ Yes	☒ No
13. "HUMAN PARTICIPANTS – IDENTIFIABLE (participants are identified or can be identified): survey questionnaires/ INTERVIEWS / focus groups / experiments / observation studies/ evaluation studies.	☐ Yes	☒ No

14. ¹ HUMAN TISSUE (e.g., human relevant material, e.g., blood, saliva, urine, breast milk, faecal material).	☐ Yes	☒ No
15. ¹ ILLEGAL/HARMFUL activities research (e.g., development of technology intended to be used in an illegal/harmful context or to breach security systems, searching the internet for information on highly sensitive topics such as child and extreme pornography, terrorism, use of the DARK WEB, research harmful to national security).	☐ Yes	☒ No
16. ¹ PERMISSION is required to access premises or research participants.	☐ Yes	☒ No
17. ¹ PERSONAL DATA PROCESSING (Any activity with data that can directly or indirectly identify a living person). For example, data gathered from interviews, databases, digital devices such as mobile phones, social media or internet platforms or apps with or without individuals'/owners' knowledge or consent, and/or could lead to individuals/owners being IDENTIFIED or SPECIAL CATEGORY DATA (GDPR ¹) or CRIMINAL OFFENCE DATA. ¹Special category data (GDPR- Art.9): "personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, or trade union membership, and the processing of genetic data, biometric data for the purpose of uniquely identifying a natural person, data concerning health or data concerning a natural person's sex life or sexual orientation".	☐ Yes	☒ No
18. ¹ PUBLIC WORKS DOCTORATES: Evidence of permission is required for use of works/artifacts (that are protected by Intellectual Property (IP) Rights, e.g. copyright, design right) in a doctoral critical commentary when the IP in the work/artifacts jointly prepared/produced or is owned by another body.	☐ Yes	☒ No
19. ¹ RISK OF PHYSICAL OR PSYCHOLOGICAL HARM (e.g., TRAVEL to dangerous places in your own country or in a foreign country (see https://www.gov.uk/foreign-travel-advice), research with NGOs/humanitarian groups in conflict/dangerous zones, development of technology/agent/chemical that may be harmful to others, any other foreseeable dangerous risks).	☐ Yes	☒ No
20. ¹ SECURITY CLEARANCE – required for research.	☐ Yes	☒ No
21. ¹ SENSITIVE TOPICS (e.g., anything deeply personal and distressing, taboo, intrusive, stigmatising, sexual in nature, potentially dangerous, etc).	☐ Yes	☒ No

M – Minimal Risk; X – More than Minimal Risk; H – High Risk

If you have answered 'Yes' to ANY of the items in the table, your application **REQUIRES** ethical review and approval using the online **Ethics Monitor** **BEFORE** commencing your research. Please apply for ethical review and approval using the **Ethics Monitor**. Further information such as links to access **Ethics Monitor** & training videos is available at MyLearning(Middlesex Online Research Ethics) area: <https://mdx.mrooms.net/course/view.php?id=12277> (Login required).

If you have answered 'No' to ALL of the items in the table, your application is Low Risk and you may NOT require ethical review and approval using **Ethics Monitor** before commencing your research. Your research supervisor will confirm this below.

Evaluation Plan / Analysis Plan

Before proceeding with the signature, provide a small but concise write-up of about 400 to 700 words outlining Evaluation Plan / Analysis Plan for your project. This plan should entail the strategies, methodologies, and approaches you will employ to assess and analyse the outcome of the project upon its Development or Research Analysis phase is finalised. Do add references to backup the arguments. This will help the supervisor assess your Ethics screening form.

Overview of Evaluation Plan / Analysis Plan

This project adopts a controlled experimental evaluation framework to assess the effectiveness of similarity measurement techniques within an academic recommendation system. The study does not involve human participants, behavioural data, surveys, or interviews. Instead, it relies exclusively on publicly available academic metadata, including research paper titles and abstracts. Consequently, the project presents minimal ethical risk and does not involve personal or sensitive data.

The primary objective of the evaluation phase is to compare two similarity paradigms: lexical similarity and semantic similarity. Lexical similarity is computed using statistical term-frequency representations such as TF-IDF combined with cosine similarity, which measure surface-level textual overlap between documents (Manning, Raghavan and Schütze, 2008). In contrast, semantic similarity uses contextual embedding models that aim to capture conceptual meaning beyond exact word matching (Cohan et al., 2020; Singh et al., 2023). The evaluation seeks to determine how these approaches differ in their ability to retrieve relevant academic documents.

The evaluation will be conducted through a structured experimental pipeline. First, a dataset of publicly accessible academic articles will be pre-processed to extract textual features. Two document representation strategies will be implemented: a lexical TF-IDF representation and a semantic embedding-based representation. Both approaches will operate within the same controlled recommendation framework to ensure comparability. Performance will be assessed using established information retrieval and recommender system evaluation metrics. These include Precision@k, Recall@k, and F1-score, which measure the relevance of recommended documents within the top-k results (Ricci, Rokach and Shapira, 2015). Ranking-based evaluation measures may also be used to assess ordering quality. Such metrics are widely accepted within recommendation system research and allow objective performance comparison (Zhang et al., 2023).

Ground truth relevance will be approximated using citation relationships or known document clustering within the dataset, consistent with evaluation practices in scholarly recommendation research (Kreutz and Schenkel, 2022). The evaluation will also consider diversity and coverage of recommendations to ensure that semantic methods do not merely increase similarity scores but improve meaningful retrieval quality.

All experiments will be reproducible and conducted within a controlled computational environment. No personal data will be processed, and no user profiling will occur. The project therefore meets institutional low-risk ethical criteria.

Overall, this evaluation plan ensures methodological rigour, transparency, and alignment with established academic standards in recommendation system research.

References:

- Cohan, A. et al. (2020) 'SPECTER: Document-level representation learning using citation-informed transformers', *Proceedings of ACL 2020*, pp. 2270–2282.
- Kreutz, C.K. and Schenkel, R. (2022) 'Scientific paper recommendation systems: a literature review of recent publications', *International Journal on Digital Libraries*, 23(4), pp. 335–369.
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Ricci, F., Rokach, L. and Shapira, B. (2015) *Recommender Systems Handbook*. 2nd edn. Springer.
- Singh, A. et al. (2023) 'SciRepEval: A multi-format benchmark for scientific document representations', *EMNLP 2023*.
- Zhang, Z. et al. (2023) 'Scholarly recommendation systems: a literature survey', *Knowledge and Information Systems*, 65, pp. 4433–4478.

Do not amend before use



[Empty rectangular box for student signature]

Student Signature: [Signature] Date: 18/02/2026

To be completed by the supervisor:

Based on the details provided in the self-assessment form, I confirm that:	Insert Y or N
The study is Low Risk and does not require ethical review & approval using Ethics Monitor	<u>Y</u>
The study requires ethical review and approval using Ethics Monitor	<u>N</u>

Supervisor Signature: [Signature] Date: 18/02/2026